

Individuelle praktische Arbeit

EcoHub Testautomation und Mapping

15. Mai 2025



Version: 0.0.5

Execution period: 06.05.2025 - 21.05.2025

Kandidat

Kento Puleo
p.kento@bluewin.ch

Berufsbildner

Sascha Listner
sascha.listner@pax.ch

Fachkraft

Kushtrim Mjeku
kushtrim.mjeku@pax.ch

Hauptexperte

Janick Wüthrich
janick.wuethrich@gmail.com

Nebenexperte

Ralf Horstmöller
ralf.horstmoeller@roche.com

DOKUMENTENVERWALTUNG

Autor: Kento Puleo
Version: 0.0.5
Datum: 15. Mai 2025
Status: In Bearbeitung
Dateiname: ipa_kp_pax_2025.pdf

Version	Datum	Änderung
0.0.1	30.04.2025	Initialisierung des $\text{\LaTeX}$ Dokuments
0.0.2	06.05.2025	Beginn der IPA
0.0.3	07.05.2025	Dokumentation der Analyse
0.0.4	08.05.2025	Dokumentation der Designphase
0.0.5	09.05.2025	

Inhaltsverzeichnis

I	Obligatorischer Teil	1
1	Aufgabenstellung im Originaltext	2
1.1	Titel der Arbeit	2
1.2	Ausgangslage	2
1.3	Detaillierte Aufgabenstellung	3
1.3.1	Mapping-Funktion	3
1.3.2	Testautomatisierung	3
1.4	Mittel und Methode	4
1.4.1	Entwicklungsumgebung	4
1.4.2	Versionierungsverwaltung	4
1.5	Vorkenntnisse	4
1.5.1	Frameworks und Libraries	4
1.5.2	Programmiersprachen	5
1.5.3	Versionierungsverwaltung	5
1.6	Vorarbeiten	5
1.7	Neue Lerninhalte	5
1.7.1	Unternehmen	5
1.7.2	Persönlich	5
1.8	Arbeiten in den letzten sechs Monaten	5
1.9	Individuelle Kriterien	6
2	Projektmethode	7
2.1	Interne Projektmethode	7
2.2	Gewählte Projektmethode	7
2.2.1	Die Wasserfallmethode	7
2.2.2	Die Phasen der Wasserfallmethode genauer erklärt	8
2.3	Begründung der Wahl	9
2.4	Vergleich mit agilen Methoden	9
3	Projektaufbauorganisation	11
4	Arbeitspakete	12

5 Backupkonzept	15
6 Zeitplan	16
7 Persönliches Fazit	17
8 Arbeitsjournal	18
 II Projektdokumentation	 19
9 Kurzfassung	20
9.1 Ausgangslage	20
9.2 Zielsetzung	20
9.3 Ergebnis	20
10 Initialisierung/Analyse	21
10.1 Erweiterung der Ausgangslage	21
10.1.1 IST-Stand	21
10.1.2 LDAP-Verzeichnisstruktur	22
10.1.3 Business Process Model der updateUser-Funktion (BPM)	24
10.1.4 Tabelle der zu testenden Endpunkte	24
10.1.5 Struktur der SOAP-Nachrichten	25
10.1.6 Analyse der bestehenden CI/CD-Pipeline	27
10.1.7 Darstellung der Systemstruktur im IST-Stand	28
10.1.8 Problemanalyse	32
10.1.9 Chancen und Risiken	32
10.1.10 Zielsetzung	33
10.1.11 Abgrenzung	33
10.2 Erweiterte Aufgabenstellung	33
10.3 Anforderungen	33
10.3.1 Funktionale Anforderungen	33
10.3.2 Nicht-funktionale Anforderungen	34
10.4 Rahmenbedingungen und Annahmen	35
11 Planung/Design	36
11.1 Zielbild und Anforderungen	36
11.2 Komponentendesign: Erweiterung von updateUser	37
11.2.1 Konfigurationsdesign: YAML-Mapping	37
11.3 Testdesign und API-Abdeckung	38
11.4 Testfälle	39
11.4.1 Vollständige XML-Payloads zu addUser, updateUser	39

11.4.2 Endpoint: addUser	40
11.4.3 Endpoint: updateUser	44
11.4.4 Endpoint: deleteUser	47
11.4.5 Endpoint: getUser	50
11.4.6 Endpoint: addUser, updateUser, deleteUser, getUser	52
11.4.7 Erweiterung der CI/CD-Pipeline mit Docker und Playwright	52
11.5 Technische Arbeitspakete	53
11.6 Tools und Technologien	54
11.6.1 Testen der Mapping-Funktion mit SoapUI und OpenLDAP	54
11.6.2 Automatisierte API-Tests mit Playwright	55
11.6.3 Versionierung	55
11.7 Clean Code Prinzipien und Namenskonzept	56
12 Implementierung	57
12.1 Mapping-Funktion	57
12.1.1 Konfigurationsdatei	57
12.1.2 Entfernen des Nutzers aus bestehenden Gruppen	58
12.1.3 Hinzufügen des Nutzers zu den neuen Gruppen	61
12.1.4 Einbindung der Mapping-Funktionen in updateUser	65
12.2 Testautomation	66
13 Test/Qualitätssicherung	67
13.1 Mapping-Funktionen	67
13.2 Testautomation	67
14 Ergebnisse	68
14.1 Mapping-Funktionen	68
14.2 Testautomation	68
15 Diskussion	69
15.1 Vergleiche	69
III Anhang	70
Hilfsmittelverzeichnis	71
Abkürzungsverzeichnis	72
Glossar	73
Abbildungsverzeichnis	74

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Tabellenverzeichnis	75
----------------------------	-----------

Quellenverzeichnis	76
---------------------------	-----------

Teil I

Obligatorischer Teil

KAPITEL 1

AUFGABENSTELLUNG IM ORIGINALTEXT

Im Folgenden Kapitel wird die Detaillierte Aufgabenstellung aus dem PkOrg kopiert und formatiert, eine erweiterte Version kann im zweiten Teil der Dokumentation gefunden werden (9.1 Erweiterte Ausgangslage & 9.2 Erweiterte Aufgabenstellung).

1.1 Titel der Arbeit

Implementierung von Mapping-Funktionen und Testautomatisierung-API im EcoHub-Service

1.2 Ausgangslage

EcoHub ist eine Plattform, die die Zusammenarbeit zwischen Makler/Brokern und Versicherungen erleichtert. Seitens Pax nutzen wir dabei die Funktionen Single-Sign-On (SSO) und User-Provisioning. Makler melden sich bei EcoHub an und können über diese Plattform auf verschiedene Versicherungen zugreifen, mit denen Sie eine Service-Vereinbarung haben.

Das geschieht folgendermassen:

1. Die Pax ist als Nutzer bei EcoHub angemeldet.
2. Broker, privat oder einer Organisation, melden sich auch als Nutzer auf EcoHub an.
3. Der Broker und die Pax können eine Service-Vereinbarung treffen. Dazu müssen beide Parteien der Vereinbarung zustimmen.
4. Nach der Vereinbarung werden die Broker Daten an die Pax gesendet. Diese prüft, ob die E-Mail des Brokers gespeichert und bekannt ist.

5. Sollte die E-Mail bekannt sein, wird eine IDP-Nummer von EcoHub angefragt. Diese wird dem Benutzer zugeteilt.
6. Sollte die E-Mail nicht bekannt sein, wirft die Pax eine Fehlermeldung an EcoHub.
7. Nachdem die IDP-Nummer dem Broker zugeteilt wurde, kann dieser über die Plattform auf die Pax Seite zugreifen, ohne sich erneut anmelden zu müssen.

1.3 Detaillierte Aufgabenstellung

Im Rahmen der IPA werden zwei Aspekte des EcoHub-Services weiterentwickelt, um dessen Funktionalität und Zuverlässigkeit zu verbessern. Einerseits wird eine Mapping-Funktion implementiert, die die automatische und präzise Zuordnung von Nutzern zu Applikationsgruppen ermöglicht. Andererseits wird eine umfassende Testautomatisierung auf API-Ebene eingeführt, um die Qualität der Schnittstellen systematisch zu prüfen und Fehler frühzeitig zu erkennen.

Beide Massnahmen zielen darauf ab, den Betrieb des EcoHub-Services effizienter zu gestalten, die Wartung zu vereinfachen und eine stabile Systemumgebung zu gewährleisten. Die nachfolgenden Abschnitte erläutern die Details zu den einzelnen Implementierungen.

1.3.1 Mapping-Funktion

Die Mapping-Funktion sorgt dafür, dass neu angelegte und bestehende Nutzer ihrer Applikationsgruppen zugeordnet werden. Diese Zuordnungen werden im OpenLDAP gespeichert und enthalten die eindeutige Kennnummer jedes Nutzers. So wird eine Präzise und nachhaltige Zuordnung gewährleistet.

Testkonzept der Mapping-Funktion

Um die Funktionalität der Mapping-Funktion sicherzustellen, wird diese umfassend getestet. Dazu gehören:

- Unit-Tests, um einzelne Komponenten isoliert zu prüfen
- Component-Tests, um die Zusammenarbeit der Module zu evaluieren
- Integration-Tests, die die Funktionalität im Gesamtsystem überprüfen

Diese Teststufen gewährleisten, dass die Mapping-Funktion fehlerfrei und stabil im Produktivbetrieb läuft.

1.3.2 Testautomatisierung

API-Tests

Der EcoHub-Service umfasst vier Schnittstellen, die künftig durch Playwright-API-Tests systematisch geprüft werden. Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Bei

fehlschlagenden Tests wird eine Benachrichtigung an die zuständigen Personen versendet, um eine zeitnahe Problembeseitigung zu gewährleisten.

Testumgebung

Die Durchführung der Tests erfolgt sowohl in der Testumgebung innerhalb der Continuous Deployment (CD) Pipeline als auch lokal. Für die lokale Ausführung wird ein embedded LDAP eingesetzt, das eine flexible und unabhängige Testumgebung ermöglicht.

Testdaten

Um die API-Tests zu unterstützen, werden spezifische Testdaten erstellt:

- Für das lokale Testen werden die Testdaten direkt im Repository hinterlegt
- In der CD-Pipeline wird zusätzlich ein Test-User im bestehenden LDAP-System angelegt, um die Funktionalität unter möglichst realistischen Bedingungen zu prüfen

1.4 Mittel und Methode

1.4.1 Entwicklungsumgebung

- | | |
|----------------------|-------------------------|
| • Lenovo ThinkPad X1 | Kenntnisse seit 2 Jahre |
| • Windows 11 | Kenntnisse seit 3 Jahre |
| • IntelliJ Ultimate | 3 Jahre |

1.4.2 Versionierungsverwaltung

- | | |
|----------|-------------------------|
| • GitHub | Kenntnisse seit 3 Jahre |
|----------|-------------------------|

1.5 Vorkenntnisse

1.5.1 Frameworks und Libraries

- | | |
|--------------|---------------------------|
| • Playwright | Kenntnisse seit 1/2 Jahre |
| • SpringBoot | Kenntnisse seit 2 Jahre |

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



1.5.2 Programmiersprachen

- TypeScript
- Java

Kenntnisse seit 3 Jahre

Kenntnisse seit 3 Jahre

1.5.3 Versionierungsverwaltung

- GitHub

Kenntnisse seit 3 Jahre

1.6 Vorarbeiten

Um eine gute und effiziente IPA durchführen zu können, wurden gewisse Vorarbeiten geleistet, welche es ermöglichte, Wissen und Erfahrungen zu gewinnen.

- Dokument Vorlage
- Einarbeitung in die Tools

1.7 Neue Lerninhalte

1.7.1 Unternehmen

Ziel ist es, Fehler frühzeitig zu erkennen und die Zuverlässigkeit der Schnittstellen sicherzustellen. Ausserdem soll die Applikationsgruppenzuweisung zukünftige Arbeiten erleichtern und den Nutzern die gewünschten Applikationen zur Verfügung stellen.

1.7.2 Persönlich

- Der Umgang und die Erfahrung Tests auf API-Ebene zuschreiben und Automatisieren
- Repetition und Erweiterung von Java und Springboot wissen

1.8 Arbeiten in den letzten sechs Monaten

Die letzten 6 Monate wurden in verschiedene Themen investiert. Folgend die Themen die behandelt wurden und die passende Zeitspanne.

- August & September: Vergleich und PoC RxJS zu Angular Signals
- Oktober & November: Testautomatisierung Playwright E2E Pilotversuch
- Dezember: Bekanntmachung mit IPA Thema

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



- Januar & Februar: Testautomatisierung API im Brokerportal

1.9 Individuelle Kriterien

Beschreibung der individuellen Kriterien.

KAPITEL 2

PROJEKTMETHODE

In diesem Kapitel wird die Wahl der Projektmethodik für die vorliegende Arbeit dargestellt. Zunächst wird auf die intern üblicherweise eingesetzte Projektmethode eingegangen. Anschliessend wird die für dieses Projekt gewählte Vorgehensweise vorgestellt und mit alternativen Methoden verglichen. Abschliessend wird die Entscheidung für die gewählte Methode begründet.

2.1 Interne Projektmethode

In den Softwareentwicklungsteams der Pax wird auf agile Projektmethoden gesetzt. Dabei kommt eine Kombination aus Scrum und Kanban zum Einsatz. Die Projekte werden in Sprints unterteilt, die wiederum in verschiedene Phasen gegliedert sind. Auch typische Scrum-Elemente wie Reviews und Retrospektiven werden regelmässig durchgeführt.

Zur Steuerung des Arbeitsprozesses wird jedoch ein Kanban-Board verwendet, um die Aufgaben zu visualisieren und den Fortschritt zu verfolgen. So lässt sich nachvollziehen, wer an welcher Aufgabe arbeitet und wie viel Zeit in jede einzelne Aufgabe investiert wird.

Ein Sprint dauert in der Regel etwa drei Wochen und beginnt mit einer Planungsphase. Es folgt eine Umsetzungsphase, die schliesslich von einer Testphase abgelöst wird. Am Ende eines Sprints erfolgt eine Reflexion, in der Erfahrungen und Erkenntnisse gesammelt sowie dokumentiert werden.

2.2 Gewählte Projektmethode

2.2.1 Die Wasserfallmethode

Bei der Wasserfallmethode handelt es sich um ein klassisches Projektmanagement-Modell, das besonders in der Softwareentwicklung weit verbreitet ist. Sie folgt einem linearen, sequenziellen Ansatz, bei dem das

Projekt in klar abgegrenzte Phasen unterteilt wird. Jede Phase muss vollständig abgeschlossen sein, bevor die nächste beginnt, wodurch der gesamte Projektverlauf strukturiert und gut planbar ist.

Das untenstehende Diagramm des Wasserfallmodells veranschaulicht diesen Ablauf deutlich: Es zeigt, wie die Phasen Anforderungsanalyse, Planung, Umsetzung, Test und Abschluss nacheinander durchlaufen werden. In der Regel erfolgt der Fortschritt nur in eine Richtung - ähnlich wie Wasser, das einen Wasserfall hinabfließt. Dennoch kann es in Ausnahmefällen notwendig sein, zu einer vorherigen Phase zurückzukehren, etwa um entdeckte Fehler oder Fehleinschätzungen zu korrigieren.

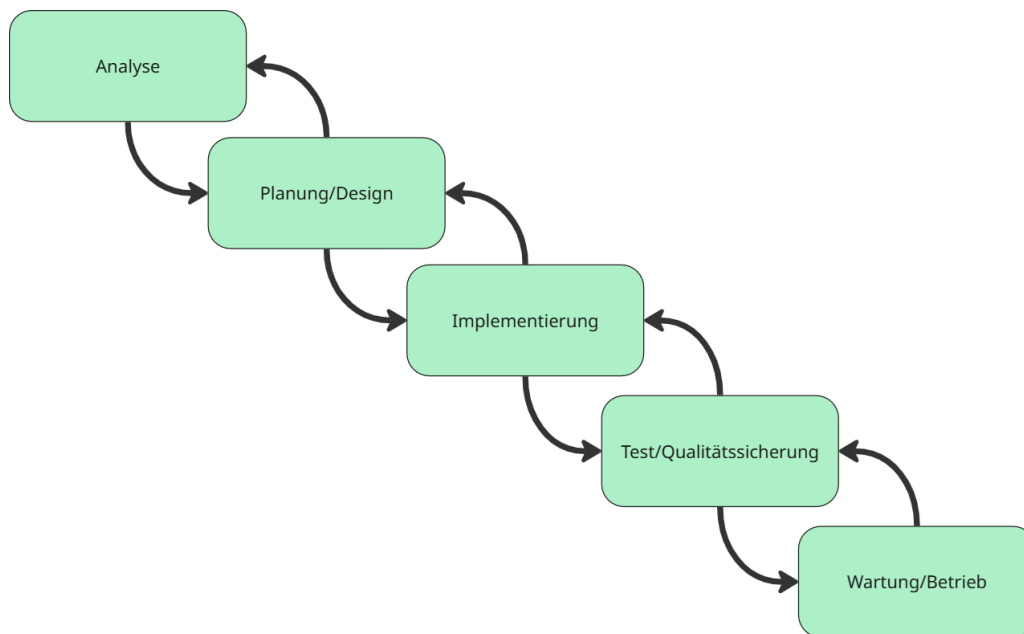


Abbildung 2.1: Wasserfallmodell (Quelle: Eigenerarbeitung)

2.2.2 Die Phasen der Wasserfallmethode genauer erklärt

Analyse

In dieser initialen Phase werden sämtliche funktionalen und nicht-funktionalen Anforderungen erhoben und dokumentiert. Gemeinsam mit allen Stakeholdern werden diese dokumentiert, der Projektumfang definiert und die Ziele festgelegt.

Planung/Design

Auf Basis der Anforderungen wird die Systemarchitektur konzipiert. Dabei werden alle Komponenten und ihre Schnittstellen definiert, die zu verwendenden Technologien und Frameworks festgelegt sowie Datenmodelle und Datenbankschemas erstellt.

Implementierung

In der Implementierungsphase erfolgt die eigentliche Programmierung der Software gemäß der Design-

Spezifikation. Die Entwickler erstellen Code nach definierten Standards, implementieren Unit-Tests und dokumentieren den Quellcode entsprechend der Vorgaben.

Test/Qualitätssicherung

Nach der Fertigstellung der einzelnen Komponenten werden diese zusammengeführt und integriert. Durch umfangreiche Tests wird die Gesamtfunktionalität validiert. Auftretende Fehler werden behoben und die Leistung des Systems optimiert.

Betrieb/Wartung

In dieser Phase wird das entwickelte System dem Kunden vorgestellt und zur abnahme bereitgestellt. Bei internen entwicklungen wird die Funktion dem Auftraggeber, meist Buissnesengineer oder Productowner, vorgestellt. Nach der Vorstellung wird die Funktion in die nächste Umgebung deployed und von den entsprechenden Personen abgenommen.

2.3 Begründung der Wahl

Die Wasserfallmethode wurde gewählt, da die Anforderungen im Vorfeld bereits klar definiert wurden und sich sehr wahrscheinlich nicht ändern werden. Es handelt sich um ein kleines Projekt, welches von einer einzigen Person ausgeführt wird, da ist es wichtig, keine komplexe Methode zu verwenden, welche womöglich viel Zeit oder Strukturierung benötigt.

2.4 Vergleich mit agilen Methoden

Für dieses Projekt gibt es keine laufenden Rückmeldungen oder Feedbackzyklen, die eine agile Methode rechtfertigen würden. Wie in der Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden zu sehen ist.

Tabelle 2.1: Vergleich zwischen Wasserfall und agilen Methoden

Kriterium	Wasserfall	Agile Methoden
Projektgrösse	Ideal für kleine, überschaubare Projekte	Gut für grosse, komplexe Projekte
Anforderungsänderungen	Schwierig zu implementieren	Flexibel und einfach umsetzbar
Planung	Detaillierte Vorplanung notwendig	Iterative Planung möglich
Teamgrösse	Funktioniert gut mit Einzelpersonen	Optimiert für Teams
Dokumentation	Umfangreich von Anfang an	Wächst mit dem Projekt
Risikomanagement	Risiken werden früh identifiziert	Kontinuierliche Risikobewertung

KAPITEL 3

PROJEKTAUFBAUORGANISATION

Im folgenden Kapitel wird die Projektaufbauorganisation veranschaulicht. Sie beschreibt die strukturierte Verteilung von Rollen, Aufgaben und Verantwortlichkeiten innerhalb des Projekts. Eine klar definierte Aufbauorganisation ist entscheidend für einen reibungslosen Ablauf, eine effiziente Kommunikation sowie die zielgerichtete Umsetzung der Projektziele.

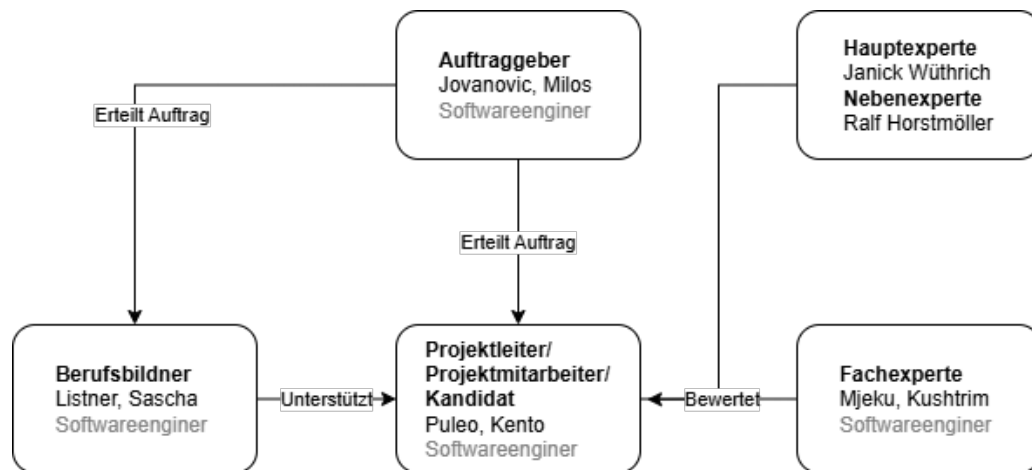


Abbildung 3.1: Projektaufbauorganisation (Quelle: Eigenerarbeitung)

In der oberen Grafik ist zu sehen welche Rolle die beteiligten Person in diesem Projekt erfüllen.

Der Auftrag wurde von einem Senior-Entwickler mit Projekterfahrung im Bereich EcoHub erteilt. Die fachliche Betreuung des Kandidaten erfolgt durch den Berufsbildner sowie einen internen Fachexperten, die ihn bereits seit etwa einem Jahr begleiten. Bei technischen Fragen steht der Berufsbildner als erste Ansprechperson zur Verfügung. Die Bewertung der Arbeit erfolgt zunächst durch den internen Fachexperten, anschliessend durch einen Haupt- und einen Nebenexperten, die extern besetzt sind.

KAPITEL 4

ARBEITSPAKETE

In diesem Kapitel werden die im Projekt definierten Arbeitspakete dokumentiert. Jedes Arbeitspaket umfasst eine eindeutige ID, einen Titel, eine Beschreibung sowie die geschätzte Bearbeitungsdauer. Die Arbeitspakete dienen der strukturierten Planung und bilden die Grundlage für die Umsetzung des Projekts. Eine Übersicht aller Arbeitspakete ist in Tabelle 4.1: Aufgabenübersicht dargestellt. Da es sich um ein Einzelprojekt handelt, erfolgt keine namentliche Zuweisung der Aufgaben.

Tabelle 4.1: Aufgabenübersicht

ID	Titel	Beschreibung	Dauer
1	Definition Arbeitspakete	Es werden alle Arbeitspakete Definiert und im entsprechenden Kapitel beschrieben.	2h
2	Erfassung der Arbeitspakete im Zeitplan	Die Vorlage für den Zeitplan wird genutzt um alle Arbeitspakete zu erfassen und messbar zu machen. Es werden alle Meilensteine definiert und im Zeitplan erkennbar gemacht. Der Zeitplan wird daraufhin in die Dokumentation implementiert.	1h
3	Erarbeitung der Aufgabenstellung Originaltext	Das Kapitel Aufgabenstellung Originaltext und alle dazu gehörigen Unterkapitel werden aus dem PkOrg Kopiert und entsprechenden erweitert.	2h
4	Festlegung der Projektmethode	Es wird eine Projekt passende Projektmethode gewählt. Es wird dokumentiert weshalb diese Methode gewählt wurde und ein Vergleich mit weiteren Methoden gemacht.	2h

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Fortsetzung von Tabelle 4.1

ID	Titel	Beschreibung	Dauer
5	Projektaufbauorganisation definieren und dokumentieren	Die Projektaufbauorganisation wird anhand einer Grafik und passendem Text im Dokument erfasst.	1h
6	Backupkonzept definieren und dokumentieren	Es wurde ein Backupkonzept gewählt, dokumentiert und angewandt	
7	Anforderungserhebung Rollen-Mapping	Ermittlung und Dokumentation der Anforderungen für die Mapping-Funktionalität. Es wird geklärt, welche externen Rollen existieren, wie diese intern verwendet werden sollen und welche Sonderfälle es gibt.	
8	Anforderungserhebung API-Testautomatisierung	Erhebung der Anforderungen für automatisierte API-Tests. Welche Endpunkte sollen getestet werden, welche Testarten sind relevant und wie sollen die Ergebnisse verwendet werden.	
9	Dokumentation der funktionalen und nicht-funktionalen Anforderungen	Zusammenführung und Ausarbeitung aller Anforderungen im entsprechenden Kapitel. Enthält Ziele, Systemgrenzen, funktionale und nicht-funktionale Anforderungen sowie Abgrenzungen.	
10	Priorisierung der Anforderungen	Die Anforderungen werden analysiert und die folge der Aufgaben bestimmen.	
11	Entwurf der Mapping-Architektur	Design der technischen Umsetzung für das Rollen-Mapping. Festlegung der Datenquellen, Mapping-Struktur (z.B. YAML), Verarbeitungsschritte und Integration in die bestehende Architektur.	
12	Entwurf der API-Teststruktur	Design der Testarchitektur für die API-Automatisierung. Auswahl von Tools, Aufbau der Testfälle, Verzeichnisstruktur, Wiederverwendbarkeit, Testdatenkonzept.	
13	Definition und Dokumentation der API-Testfälle	Detaillierte Ausarbeitung aller Testfälle für die zu testenden API-Endpunkte. Die Testfälle werden so beschrieben, dass sie direkt als Basis für automatisierte Tests verwendet werden können. Sie umfassen Ziel, Eingaben, erwartetes Verhalten, Vorbedingungen und Testart.	

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Fortsetzung von Tabelle 4.1

ID	Titel	Beschreibung	Dauer
14	Grobentwurf einer Lösung erstellen	Es wurde ein Grobentwurf der beiden Lösungen erstellt und dokumentiert	
15	Implementierung der Mapping-Funktion	Umsetzung der Mapping-Logik zur Zuordnung von externen Rollen auf interne LDAP-Gruppen. Implementierung wird laufend dokumentiert.	
16	Testen der Mapping-Funktion	Die Mapping-Funktion wurde ausgiebig getestet und die Funktionen wurden mit passenden Tests ausgestattet. Umsetzung wird laufend dokumentiert	
17	Implementierung der API Tests	Umsetzung der API Tests anhand der erstellten Testfällen. Implementierung wird laufend dokumentiert.	
18	Testen der API Tests	Die API-Tests werden auf Erfolg getestet und Fehler entsprechend behoben. Umsetzung wird laufend dokumentiert.	
19	Anbindung der Tests an CI/CD	Die API und Funktionstests sind in der Pipeline eingebunden und laufen lokal wie auch auf den Umgebungen. Umsetzung wird laufend dokumentiert.	
20	Kurzfassung des Dokuments erfassen	Die Arbeit wurde als Kurzfassung geschrieben mit 3 Kapiteln.	
21	Persönliches Fazit im Dokument festhalten	Das Persönliche Fazit wurde im Dokument erfasst.	
22	Überprüfen des Dokuments	Das Dokument wird gründlich durchgelesen und auf Fehler in der Formatierung wie aber auch in der Rechtschreibung geprüft.	

KAPITEL 5

BACKUPKONZEPT

Während dieser Arbeit wird die 3-2-1-1-0-Methode verwendet. Diese ist eine moderne Erweiterung der klassischen 3-2-1-Methode. In der 3-2-1-Methode stehen die einzelnen Zahlen für folgende Regeln.

3. Mindestens drei Kopien
2. Auf mindestens zwei verschiedenen Medientypen. (z.B. Lokale Festplatte und Cloud)
1. Und mindestens eine Kopie muss an einem geografisch anderen Standort sein.

Das Ziel dieser Methode ist es, einerseits eine sichere Datenenerhältlichkeit zu gewährleisten und ebenfalls bei Problemen keine Daten zu verlieren.

In der modernen 3-2-1-1-0-Methode werden zwei neue Regeln hinzugefügt.

1. Mindestens eine Kopie sollte offline sein, also nicht mit dem Netzwerk verbunden und unveränderlich.
0. Die Kopien dürfen keine Fehler aufweisen und werden dafür regelmässig getestet.

Mit den neuen zwei Regeln werden die Daten nun auch gegen Ransomware oder menschliche Fehler geschützt. Auch das Testen der Backups wird gefordert, um ungewünschte Überraschungen zu vermeiden.

KAPITEL 6

ZEITPLAN

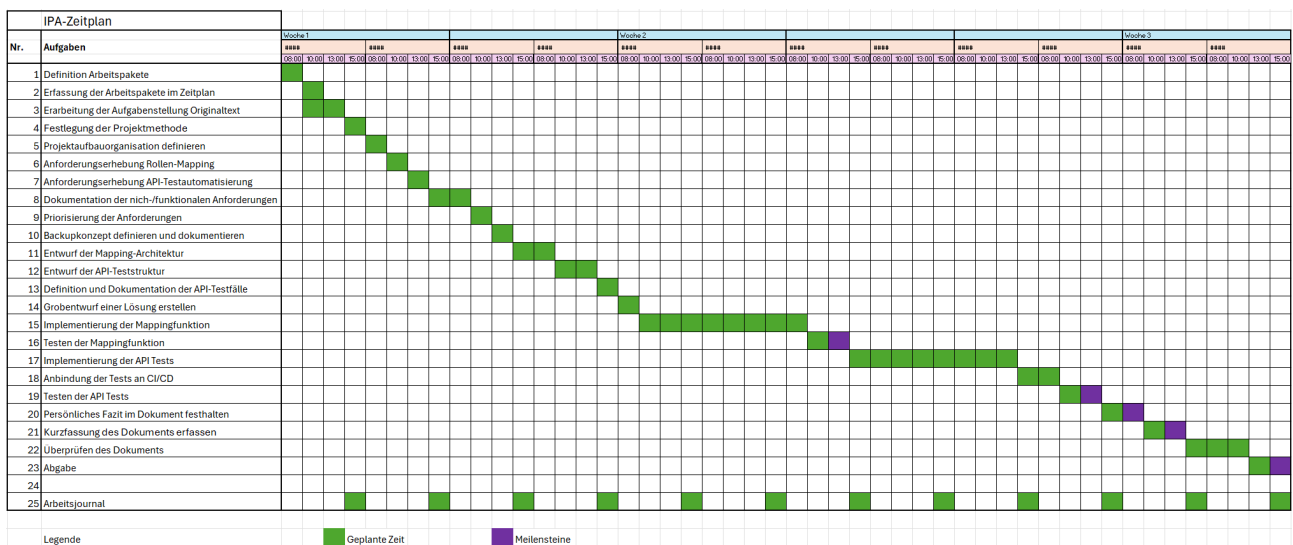


Abbildung 6.1: Rollenmodell auf der EcoHub-Plattform (Quelle: Eigene Darstellung)

KAPITEL 7

PERSÖNLICHES FAZIT

Persönliche Schlussfolgerungen und Erkenntnisse.

KAPITEL 8

ARBEITSJOURNAL

Das Arbeitsjournal dient der Nachvollziehbarkeit der Arbeiten, welche Kento Puleo zwischen dem 06.05.2025 und dem 21.05.2025 getätigt hat. Ebenso dient es zur Reflexion.

Jedes Arbeitsjournal ist gleich aufgebaut. Zuerst werden alle Tätigkeiten aufgeschrieben und Probleme oder Erfolge vermerkt. In einem weiteren Text wird eine persönliche Reflexion der Arbeit wiedergegeben. Zum Schluss wird in einer Tabelle aufgeführt, welche Arbeitspakete an diesem Tag erledigt werden konnten.

Jedes Arbeitsjournal wird vom Kandidaten (Kento Puleo) und dem Berufsbildner (Sascha Listner) unterschrieben. Das Arbeitsjournal wird täglich geführt.

Das Arbeitsjournal ist im Anhang zu finden.

Teil II

Projektdokumentation

KAPITEL 9

KURZFASSUNG

Kurze Zusammenfassung der Arbeit.

9.1 Ausgangslage

Beschreibung der Ausgangssituation.

9.2 Zielsetzung

Beschreibung der zu erreichenden Ziele.

9.3 Ergebnis

Beschreibung der erzielten Ergebnisse.

KAPITEL 10

INITIALISIERUNG/ANALYSE

In diesem Kapitel wird die aktuelle Ausgangslage analysiert, Schwachstellen identifiziert sowie die angestrebten Ziele und Anforderungen beschrieben.

10.1 Erweiterung der Ausgangslage

10.1.1 IST-Stand

Benutzer, die sich über die EcoHub-Plattform bei Pax anmelden möchten, müssen aktuell initial manuell über eine interne Eingabemaske im LDAP-Verzeichnis angelegt werden. Dabei wird zwischen Private Vorsorge (PV) und Berufliche Vorsorge (BV) unterschieden. Ein Broker, der durch diese Maske im LDAP erfasst wird, erhält anschliessend die notwendigen Rollen und Gruppen.

Dieser Prozess erfolgt manuell, ist fehleranfällig und verursacht hohen administrativen Aufwand.

Sollte sich ein Broker anmelden wollen, der nicht im LDAP vorhanden ist, scheitert der Vorgang, da aktuell keine automatische Benutzererstellung existiert. Die vorhandene `addUser`-Funktion generiert eine IDP-Nummer, welche für die Durchführung des Single Sign-On (SSO) zwingend erforderlich ist. Die Web Access Management (WAM) prüft im Rahmen des SSO auf das Vorhandensein dieser IDP-Nummer. Fehlt sie, wird der Zugriff verweigert.

Auf der EcoHub-Plattform existieren bereits eigene Berechtigungskonzepte. Diese stimmen grösstenteils mit jenen der Pax überein. Es wird zwischen Einzelleben (EL) und Kollektivleben (KL) unterschieden (siehe Abbildung 10.1). Obwohl diese Rollen über EcoHub zugewiesen werden können, werden sie derzeit nicht automatisch in das LDAP-Verzeichnis übernommen und operativ nicht weiterverwendet.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Einzelleben (PV) (EL_*)

Diese Rollen beziehen sich auf **Einzelverträge** der Lebensversicherung (Privatkunden).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	EL_INFO	Zugriff auf Kundendaten zu Einzelverträgen (Details, Rendite etc.).
Offertwesen	EL_OFFERT	Berechtigt zur Nutzung des Offertrechners (z. B. Erstellung neuer Angebote).
Vertragsverwaltung	EL_VERTRAG	Berechtigt zur Anpassung von Kunden- und Vertragsdaten.
Leistungsfälle	EL_LEISTUNG	Einsicht in laufende Leistungsfälle (z. B. Todesfall-Leistungen).

Kollektivleben (BV) (KL_*)

Diese Berechtigungen betreffen **Gruppenverträge**, z. B. BVG (Pensionskassen).

Rollenname	Code	Beschreibung
Bestandsdaten abrufen	KL_INFO	Zugriff auf Kundendaten zu Gruppenverträgen.
Offertwesen	KL_OFFERT	Berechtigt zur Nutzung des Offertrechners für Firmenkunden.
Vertragsverwaltung	KL_VERTRAG	Berechtigt zur Anpassung von Kollektivverträgen.
Eigene BVG-Verträge	KL_EIGENVERTRAG	Einsicht in eigene BVG-Verträge (Pensionskassenverwaltung).

Abbildung 10.1: Rollenmodell auf der EcoHub-Plattform (Quelle: Miro Board)

10.1.2 LDAP-Verzeichnisstruktur

Zur besseren Nachvollziehbarkeit der bestehenden Benutzerverwaltung wurde im Rahmen der Analysephase auch die aktuelle LDAP-Struktur untersucht. Abbildung 10.2 zeigt den lokalen Aufbau des LDAP-Verzeichnisses im Projektkontext.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo

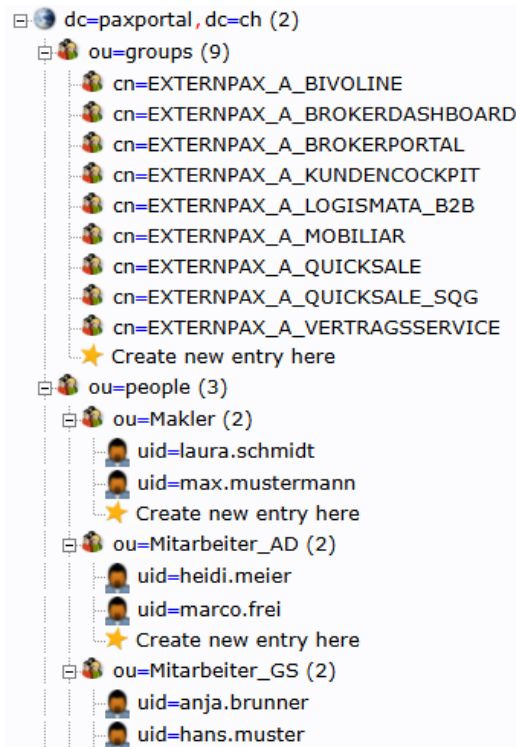


Abbildung 10.2: LDAP-Verzeichnisstruktur im lokalen Setup (Quelle: Eigene Darstellung)

Die Struktur gliedert sich wie folgt:

- **Basis-Domain:** `dc=paxportal,dc=ch`
- **Organisatorische Einheiten (OUs):**
 - `ou=groups`: Enthält LDAP-Gruppen, welche externen Rollen zugewiesen sind. Die Gruppen folgen dem Namensschema `cn=EXTERNPAX_A_<ROLLE>` und bilden verschiedene Anwendungen wie `BROKERPORTAL`, `LOGISMATA_B2B` oder `QUICKSALE` ab.
 - `ou=people`: Beinhaltet Benutzerobjekte, unterteilt in:
 - * `ou=Makler`: für Broker-Nutzer
 - * `ou=Mitarbeiter_AD`: für interne AD-Mitarbeitende
 - * `ou=Mitarbeiter_GS`: für weitere interne Mitarbeitende

Die im Bild gezeigten Benutzereinträge (`uid=laura.schmidt`, `uid=hans.muster` etc.) sind **ausschliesslich Dummy-Daten** und dienen lediglich der Veranschaulichung.

Relevanz für das Projekt

Diese LDAP-Struktur ist zentral für den *EcoHub Provisioning Service*, welcher Benutzer automatisiert erstellt, aktualisiert oder löscht. Dabei ist besonders die rollenbasierte Gruppenzuweisung entscheidend: Über ein

externes Mapping (z. B. EL_, KL_) wird gesteuert, in welche Gruppen ein Benutzer aufgenommen wird.

Datenschutz und Veröffentlichung

Vor der Integration dieser Struktur in das GitHub-Repository wurde geprüft, ob die cn-Bezeichner der Gruppen bedenkenlos veröffentlicht werden dürfen. Diese Klärung war notwendig, um sicherzustellen, dass keine vertraulichen oder geschützten Produktionsinformationen offengelegt werden. Die hier gezeigte Struktur ist vollständig anonymisiert und enthält keine personenbezogenen oder produktiven Daten.

10.1.3 Business Process Model der updateUser-Funktion (BPM)

Eine *updateUser*-Funktion ist vorhanden, erfüllt jedoch noch nicht die gewünschten Anforderungen hinsichtlich automatischer Rechtezuweisung oder Synchronisierung. Wie in der Abbildung 10.3 zu sehen ist, wird bei der updateUser-Funktion die gleiche Logik wie bei der addUser-Funktion verwendet.

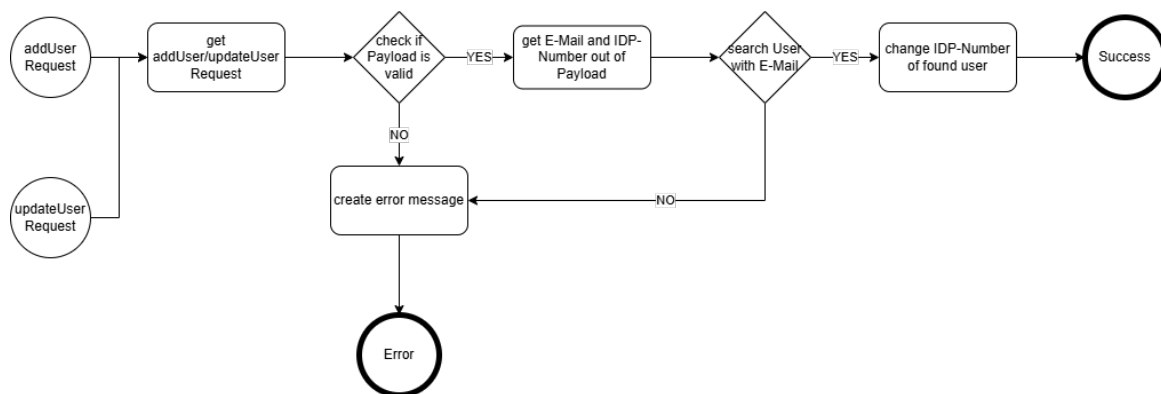


Abbildung 10.3: Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigene Darstellung)

10.1.4 Tabelle der zu testenden Endpunkte

Die folgende Tabelle gibt einen Überblick über die zu testenden Endpunkte und beschreibt deren aktuelle Funktionalitäten.

Tabelle 10.1: Übersicht der zu testenden Endpunkte

Endpoint	Beschreibung
addUser	Sucht einen Broker im LDAP-Verzeichnis anhand der E-Mail-Adresse und weist diesem eine IDP-Nummer zu.
updateUser	Aktualisiert die IDP-Nummer im LDAP-Verzeichnis.
deleteUser	Entfernt die IDP-Nummer eines Benutzers aus dem LDAP-Verzeichnis.

Fortsetzung von Tabelle 10.1

Endpoint	Beschreibung
getUserStatus	Prüft die Existenz der IDP-Nummer eines Benutzers im LDAP-Verzeichnis.

10.1.5 Struktur der SOAP-Nachrichten

Die Benutzerprovisionierung erfolgt über eine SOAP-Schnittstelle, bei der unterschiedliche Endpunkte wie `addUser`, `updateUser`, `deleteUser` und `getUserStatus` verwendet werden. Die meisten Endpunkte besitzen eine eigene XML-Struktur, welche den jeweiligen Verwendungszweck widerspiegelt. Die Struktur ist vorgegeben, wurde aber im Rahmen der Analyse hinsichtlich Relevanz und Vollständigkeit geprüft.

Endpunkt: `addUser`

Die folgende Nachricht zeigt eine typische `addUser`-Anfrage. Die wichtigsten übermittelten Informationen betreffen die Benutzeridentifikation (`idpUserId`), Rollenzuweisung (`igRoles`) sowie persönliche Angaben.

Listing 10.1: Beispielhafte `addUser`-Anfrage

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>123</idpUserId>
      <orgId>1</orgId>
      <igRoles>EL_OFFERT</igRoles>
      <orgRegNo>10</orgRegNo>
      <primaryOU>
        <id>1</id>
        <regNo>10</regNo>
        <type>test</type>
        <name>test</name>
      </primaryOU>
      <employed>true</employed>
      <givenName>max</givenName>
      <surname>mustermann</surname>
      <userEmail>max.mustermann1@email.ch</userEmail>
      <userOfficePhone>123</userOfficePhone>
      <userMobilePhone>123</userMobilePhone>
      <userBirthday>1970-01-01T00:00:00Z</userBirthday>
      <userSex>M</userSex>
      <userLanguage>DE</userLanguage>
      <validFrom>2024-01-01T00:00:00Z</validFrom>
      <validTo>2024-12-31T00:00:00Z</validTo>
      <userType>PERSONAL</userType>
      <userCategory>BROKER</userCategory>
      <creationDate>2024-01-01T00:00:00Z</creationDate>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<creationUser>admin</creationUser>
<modificationDate>2024-01-01T00:00:00Z</modificationDate>
<modificationUser>admin</modificationUser>
<requestId>1</requestId>
<requestTime>2024-01-01T00:00:10Z</requestTime>
</user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Endpunkt: updateUser

Bei der updateUser-Anfrage wird die gleiche XML-Struktur verwendet wie bei der addUser-Anfrage.

Endpunkt: deleteUser

Die folgende Nachricht zeigt eine typische deleteUser-Anfrage. Die wichtigsten übermittelten Informationen betreffen die Benutzeridentifikation (idpUserId).

Listing 10.2: Beispielhafte addUser-Anfrage

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
userProvisioningSchemaTypes-v3.0/">
<soapenv:Header/>
<soapenv:Body>
  <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
    <idpUserId>222</idpUserId>
    <deletionDate>2024-12-01T00:00:00Z</deletionDate>
    <deletionUser>admin</deletionUser>
    <requestId>1</requestId>
    <requestTime>2024-12-01T00:00:10Z</requestTime>
  </user:deleteUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Endpunkt: getUserStatus

Die folgende Nachricht zeigt eine typische getUserStatus-Anfrage.

Listing 10.3: Fehlermeldung bei addUser-Anfrage

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=123</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```


Antwortstruktur bei Fehlern

Im Erfolgsfall erfolgt keine Rückmeldung. Tritt jedoch ein Fehler auf, wird eine Antwort mit einem Fehlercode und einer Nachricht geliefert, folgend ein Beispiel:

Listing 10.4: Fehlermeldung bei addUser-Anfrage

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Processing add user failed for idpUserId=123</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Feldrelevanz

Folgende Felder sind für die weitere Verarbeitung im Provisioning-Service besonders relevant:

- **idpUserId:** Eindeutige Identifikation des Benutzers im LDAP
- **igRoles:** Grundlage für rollenbasierte Gruppenzuweisung
- **userEmail:** Wird als Mail-Attribut in LDAP übernommen
- **requestTime:** Technisches Feld zur Nachvollziehbarkeit

Die XML-Struktur bildet somit das zentrale Bindeglied zwischen dem externen Aufrufer und der internen LDAP-Provisionierung.

10.1.6 Analyse der bestehenden CI/CD-Pipeline

Zur Qualitätssicherung und Automatisierung der Build- und Testprozesse wird im Projekt eine GitHub-Actions-basierte CI/CD-Pipeline eingesetzt. Diese ist in der Datei `pipeline.yml` definiert und wird bei jedem push ins Repository ausgeführt.

Übersicht der Jobs

Die Pipeline besteht aus vier zentralen Jobs:

- **test:** Führt automatisierte Unit-Tests mit Maven aus
- **build:** Erstellt das Projekt und bereitet ein Docker-Image vor
- **publish:** Veröffentlicht das Docker-Image in eine Container Registry (nur bei Tags)
- **delete_artifacts:** Entfernt erstellte Artefakte nach erfolgreicher Veröffentlichung

Technische Details (Auszug Job test)

Listing 10.5: Auszug aus pipeline.yml

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'
      - name: Execute tests
        run: chmod +x ./mvnw && ./mvnw test
```

Die Tests werden vor dem Build ausgeführt und dienen als Quality-Gate. Nur bei erfolgreichem Testdurchlauf wird das Projekt gebaut und ein Docker-Image erzeugt.

Integration in den Entwicklungsprozess

Diese Pipeline bildet die Grundlage für die kontinuierliche Validierung und Bereitstellung des Provisioning-Dienstes. Dadurch wird sichergestellt, dass:

- Änderungen stets mit bestehenden Funktionalitäten kompatibel sind
- Fehler frühzeitig erkannt werden
- Releases nur bei validierten Git-Tags veröffentlicht werden

10.1.7 Darstellung der Systemstruktur im IST-Stand

Folgend wird die Systemstruktur im IST-Stand dargestellt. Diese wird während dieser Arbeit jedoch nicht verändert und dient nur zur Veranschaulichung des aktuellen Zustands.

Systemkontextdiagramm (C1)

Zur Veranschaulichung des IST-Zustands wurde ein C1-Diagramm nach dem C4-Modell erstellt. Dieses stellt die Systemlandschaft im Kontext dar und zeigt die Interaktionen zwischen Benutzer, zentralem Dienst (Eco-Hub), dem Provisioning Service sowie den Umsystemen.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo

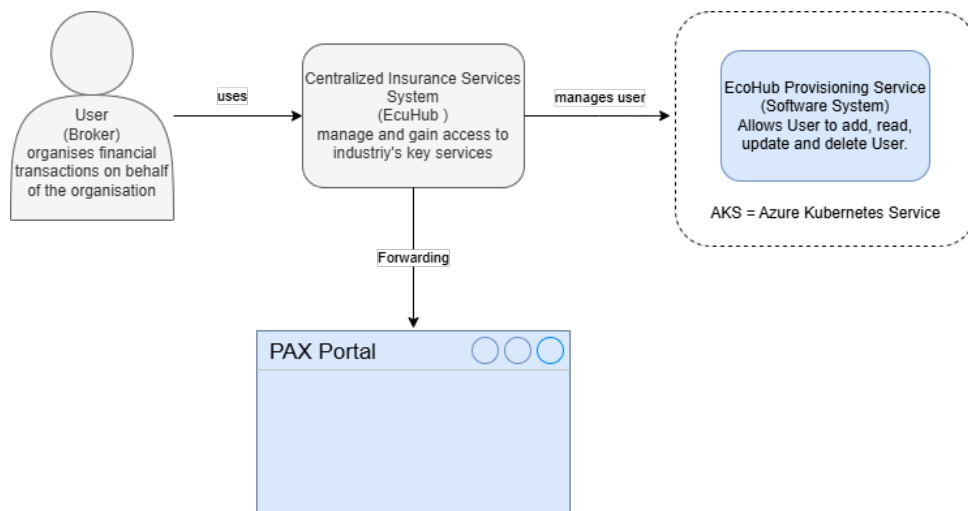


Abbildung 10.4: Systemkontext (C1) - IST-Zustand der Benutzerverwaltung (Quelle: Eigene Darstellung)

Container-Diagramm (C2)

Ergänzend zum Systemkontext (C1) wird in der folgenden Abbildung die interne Struktur des Systems im Sinne eines C2-Diagramms dargestellt. Dieses zeigt die einzelnen technischen Container, deren Hauptaufgaben sowie die Kommunikationsflüsse zwischen ihnen im aktuellen Systemzustand.

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen

Kandidat: Kento Puleo

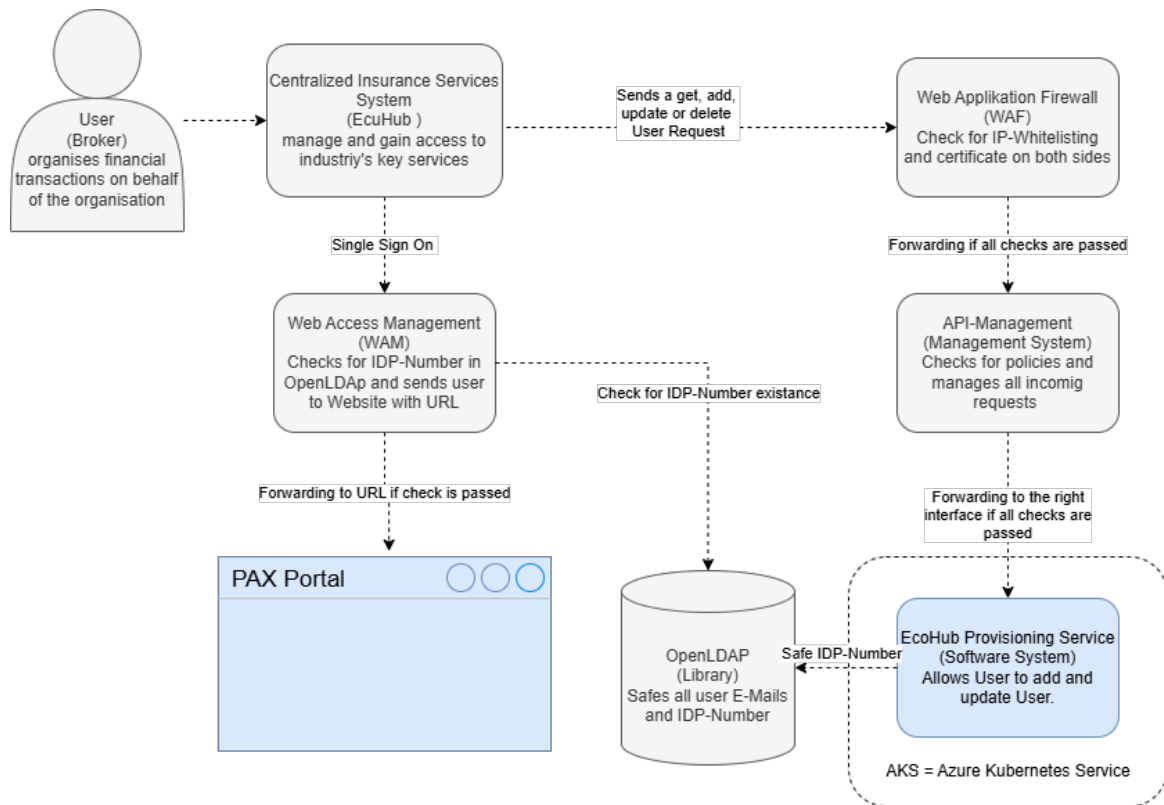


Abbildung 10.5: C2-Diagramm IST-Zustand - Containerübersicht (Quelle: Eigene Darstellung)

Die wichtigsten Container im aktuellen System sind:

- **Web Application Firewall (WAF):** Führt IP-Whitelisting sowie Zertifikatsprüfungen durch und schützt die Schnittstellen gegenüber dem Internet.
- **API-Management:** Verwaltet eingehende API-Anfragen, prüft auf Richtlinien (Policies) und leitet validierte Anfragen an die zuständigen Systeme weiter.
- **Web Access Management (WAM):** Übernimmt die zentrale Authentifizierung (SSO) und prüft anhand der IDP-Nummer im LDAP, ob ein Benutzer autorisiert ist.
- **OpenLDAP:** Verzeichnisdienst zur Speicherung von Benutzerdaten, insbesondere E-Mail-Adressen und IDP-Nummern.
- **EcoHub Provisioning Service:** Eine Spring Boot-Anwendung, bereitgestellt auf Azure Kubernetes (AKS), welche Benutzer über SOAP-Schnittstellen verwalten kann (hinzufügen, aktualisieren, löschen).
- **PAX Portal:** Die Zielplattform, auf die Broker nach erfolgreichem Login weitergeleitet werden, wenn alle Prüfungen bestanden wurden.

Die Kommunikation zwischen diesen Containern erfolgt aktuell sequenziell, mit mehreren Prüfungen und Sicherheitsbarrieren. Änderungen oder Fehler in einem der Container können sich direkt auf das Verhalten des Gesamtsystems auswirken.

Komponentendiagramm (C3) - EcoHub Provisioning Service

Das nachfolgende C3-Diagramm zeigt den internen Aufbau des **EcoHub Provisioning Service** auf Komponentenebene. Der Service ist eine Spring Boot-Anwendung. Er stellt eine SOAP-Schnittstelle zur Verwaltung von Broker-Benutzern bereit. Die Benutzerverwaltung erfolgt durch Interaktion mit einem extern angebundenen OpenLDAP-Verzeichnisdienst.

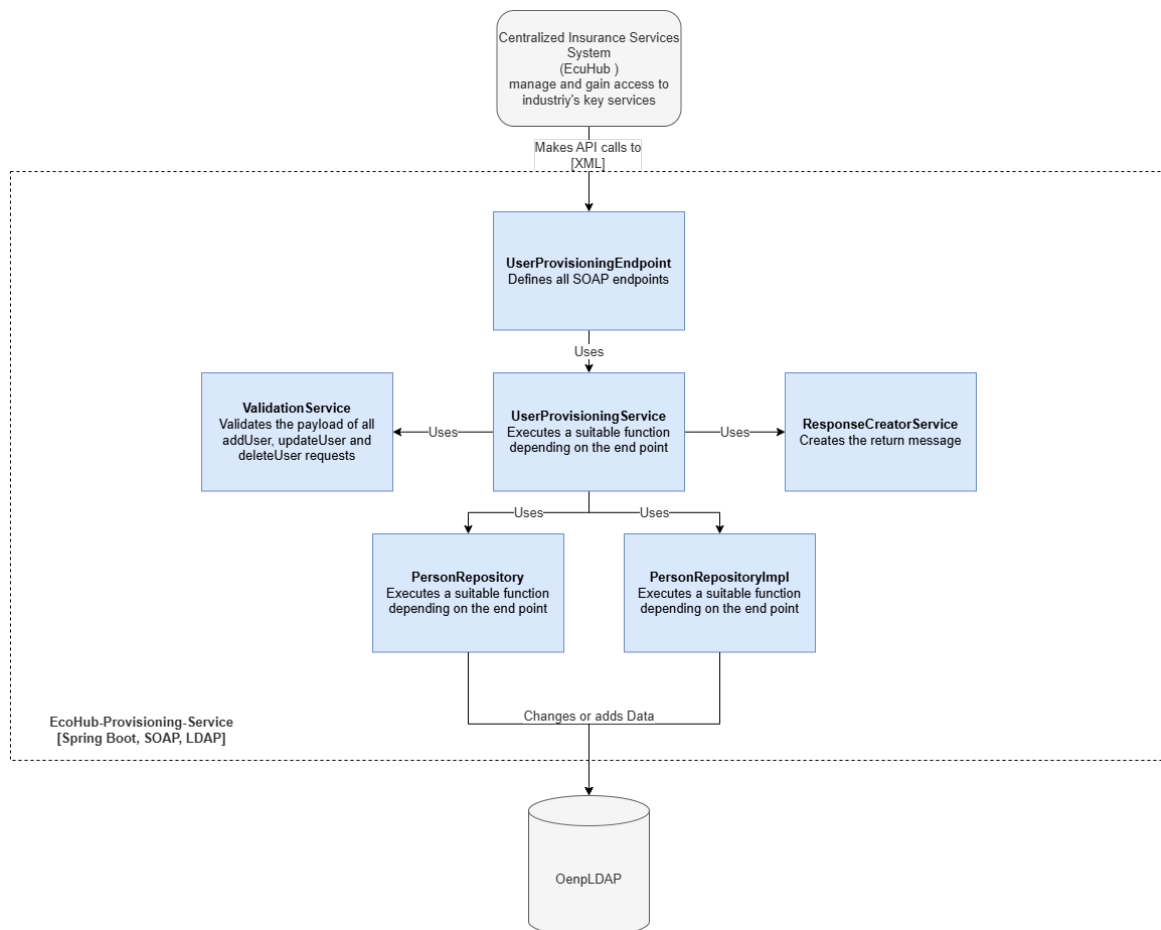


Abbildung 10.6: C3-Komponentendiagramm - EcoHub Provisioning Service (Quelle: Eigene Darstellung)

Die wichtigsten Komponenten im Diagramm sind:

- **UserProvisioningEndpoint**
SOAP-Webservice-Endpunkt, der externe Anfragen wie addUser, updateUser, deleteUser oder getUserStatus entgegennimmt.
- **UserProvisioningService**
Zentrale Geschäftslogik. Validiert die Anfragen, delegiert LDAP-Operationen und erstellt Rückgabemeldungen.

- **ValidationService**

Führt strukturierte Prüfungen der eingehenden Payloads durch (z. B. Pflichtfelder, Formate, Längenbeschränkungen) und verwendet Adapter-Objekte zur Entkopplung vom SOAP-Modell.

- **ResponseCreatorService**

Baut standardisierte Antwortobjekte (inkl. Fehlermeldungen) zur Rückgabe über SOAP.

- **PersonRepository & PersonRepositoryImpl**

Repositories zur Suche, Änderung und Löschung von Nutzerdaten im OpenLDAP-Verzeichnis. Nutzen `LdapTemplate` von Spring.

- **OpenLDAP (extern)**

Verzeichnisdienst, der Brokerdaten und ihre Rollen speichert. Wird durch den Provisioning Service gelesen und beschrieben.

- **EcoHub (extern)**

Die EcoHub-Plattform ruft den Provisioning-Service über eine definierte WSDL-Schnittstelle auf und sendet XML-basierte Requests.

Die Kommunikation erfolgt synchron via SOAP, die Daten werden direkt im LDAP-Verzeichnis persistiert. Alle Komponenten sind lose gekoppelt und durch Interfaces und Adapter abstrahiert, was Testbarkeit und Erweiterbarkeit verbessert.

10.1.8 Problemanalyse

Schwachstellen sind konkrete Mängel oder Probleme im aktuellen System oder Prozess. Sie beschreiben, was heute nicht gut funktioniert und somit den Änderungsbedarf begründet. Darunter fallen aktuell folgende Punkte:

- Der Benutzer muss manuell im LDAP eingetragen werden, was ein hoher Aufwand ist und die manuelle Bearbeitung birgt ein hohes Fehlerrisiko.
- Die Rollen von EcoHub werden nicht im LDAP übernommen, was bedeuten kann, dass Broker im EcoHub andere Berechtigungen haben.
- Es gibt keine Tests auf die Endpunkte des EcoHub-Services. Schlägt ein Endpunkt fehl oder werden Änderungen gemacht, wird der Nutzer es als erstes bemerken.

10.1.9 Chancen und Risiken

Risiken sind mögliche Probleme oder Gefahren, die beim Projektverlauf oder der Umsetzung der Änderung auftreten könnten - also zukünftige Unsicherheiten. Folgende wurden für dieses Projekt erkannt.

- Fehlerhafte Zuordnung von Rollen. Die Mapping-Funktion weist den Broker in falsche Gruppen und Rollen.

- Änderung an LDAP-Struktur. Ungründliche Vorbereitung und Arbeiten könnte die LDAP-Struktur verändern.
- Unvollständige Testabdeckung. Werden die Funktionen oder Endpunkte nicht vollständig abgedeckt, kann es weiterhin zu unentdeckten Problemen führen.

10.1.10 Zielsetzung

Nun sind verschiedene Ziele für das EcoHub vorgesehen. Der ganze Prozess soll automatisiert geschehen, das bedeutet die manuelle Erstellung der Broker über die Maske fällt weg. Der Broker soll die Möglichkeit bekommen, über das EcoHub sich bei der Pax anzumelden, insofern er eine Service-Vereinbarung eingegangen ist, und damit ein neuer Nutzer im LDAP erstellt und gespeichert wird. Bei der Erstellung wie aber auch nachträglich sollen die Berechtigungen, die der Broker auf der EcoHub-Plattform erhält, die gleichen sein wie die im LDAP. Auch die einzelnen Schnittstellen des EcoHub-Services sollen getestet werden, um Probleme vorzeitig zu erkennen und zu lösen.

10.1.11 Abgrenzung

Wichtig ist, sich nicht in der eigenen Arbeit zu verlieren oder sich in andere Aufgaben zu verlieren. Die Automatisierung des Prozesses, um einen neuen Broker hinzuzufügen, ist nicht Teil der IPA. Auch dass die Gruppen-Zuteilung bei der initialen Anmeldung geschieht, ist nicht Teil der IPA.

10.2 Erweiterte Aufgabenstellung

Die Aufgabe in dieser IPA bezieht sich auf zwei der Ziele. Es soll eine Mapping-Funktion geschrieben werden welche die EcoHub Rollen mit den internen Rollen verbindet. Das soll geschehen sollten sich die Rollen des Broker ändern und ein update nötig sein. Auch sollen alle Schnittstellen des EcoHub-Services automatisiert getestet werden, die Automation soll über eine Pipeline laufen.

10.3 Anforderungen

10.3.1 Funktionale Anforderungen

Bei Funktionalen Anforderungen handelt es sich um das (WAS?). Was soll das System oder Produkt tun, besser gesagt welche Funktionen oder Verhaltensweisen muss es bieten. Folgend die wichtigsten Funktionalen Anforderung im bezug auf die zwei Aufträge.

Mapping-Funktion

- Der Broker wird entsprechend seiner EcoHub-Rolle den korrekten Gruppen im LDAP zugeordnet.

- Gruppen, in denen der Broker nicht mehr Bestandteil ist, aus denen wird er entfernt.
- Bei nicht bekannten Rollen wird die Anfrage ignoriert und der Broker kriegt keine neuen Berechtigungen.
- Die Funktion muss das Mapping aus einer externen Konfigurationsdatei (z.B. application.yml) laden, damit Änderungen ohne Codeanpassung möglich sind.
- Die Funktion muss mehrere Rollen gleichzeitig verarbeiten können.

API-Testautomatisierung

- Alle bekannten API-Endpunkte des EcoHub-Provisioning-Services müssen durch automatisierte Tests abgedeckt werden.
- Die Tests müssen sicherstellen das konkrete Fehlermeldungen zurückgegeben werden.
- Tests müssen jederzeit wiederholbar sein, ohne manuelle Vorbereitung (z.B. Setup- und Teardown-Routinen für Testdaten).
- Die Tests müssen nicht nur wiederholbar sondern auch automatisch durchgeführt werden.

10.3.2 Nicht-funktionale Anforderungen

Bei nicht-funktionalen Anforderungen handelt es sich um das (WIE?). Wie soll das System funktionieren und welche Qualität soll dieses bieten.

Mapping-Funktion

- Die Verarbeitung einer Mapping-Anfrage soll maximal 500 ms dauern.
- Ungültige oder unbekannte Rollen verursachen einen kontrollierten Fehler mit Logging.
- Die Mapping-Logik muss durch Unit-Tests abdeckbar sein.

API-Testautomatisierung

- Die komplette Testausführung soll in unter 2 Minuten durchführbar sein, um schnelle Feedback-Zyklen zu ermöglichen.
- Für die Tests dürfen keine sensiblen Daten verwendet werden. Es werden Testdaten genutzt, welche nicht mit echten Nutzern in Verbindung gebracht werden können.
- Die Testfälle müssen gut strukturiert, benannt und dokumentiert sein, sodass sie auch von Dritten gepflegt werden können.
- Das Testsystem soll erweiterbar sein, damit neue Endpunkte mit geringem Aufwand integriert werden können.

10.4 Rahmenbedingungen und Annahmen

Rahmenbedingungen beschreiben äussere Faktoren und definieren den Spielraum, der das Projekt nicht aktiv beeinflusst, an den sich aber gehalten werden muss. Für diese Arbeit gelten folgende Rahmenbedingungen:

1. Das bestehende LDAP-System (OpenLDAP) darf nicht strukturell verändert werden (Schema, Gruppenstruktur bleibt gleich).
2. Es muss eine bestehende Spring Boot Applikation verwendet werden.
3. Es dürfen nur bereits vorhandene Gruppen im LDAP verwendet werden.
4. Die Integration erfolgt via SOAP-Schnittstelle des EcoHub.
5. Die API-Tests werden mit Playwright gemacht, da Pax weit alle API- und End-to-End-Tests mit Playwright geschrieben werden.

KAPITEL 11

PLANUNG/DESIGN

Gemäss dem Wasserfallmodell folgt auf die Analysephase die Designphase, in der die geplanten Systemanpassungen detailliert ausgearbeitet werden. Aufbauend auf den zuvor identifizierten Schwächen wird in diesem Kapitel beschrieben, wie die Benutzerverwaltung des EcoHub Provisioning Service funktional erweitert und technisch abgesichert werden soll.

Neben der Erweiterung bestehender Funktionalität steht auch die Einführung automatisierter Tests zur Qualitätssicherung im Fokus. Die folgenden Abschnitte stellen das Zielbild, die geplanten Änderungen auf Komponentenebene sowie die vorgesehenen Massnahmen zur Absicherung und Testbarkeit der Erweiterung vor.

11.1 Zielbild und Anforderungen

Im Rahmen der zweiten Projektphase werden zwei zentrale Erweiterungen am EcoHub Provisioning Service vorgenommen:

1. Die Funktion `updateUser` wird um eine automatische Gruppenzuweisung im LDAP erweitert. Die Zuweisung basiert auf den übermittelten Rolleninformationen (`igRoles`) und erfolgt konfigurierbar über ein YAML-basiertes Mapping.
2. Für alle vier vorhandenen SOAP-Endpunkte (`addUser`, `updateUser`, `deleteUser`, `getUserStatus`) werden API-Tests mit Playwright implementiert. Diese Tests werden in die bestehende CI/CD-Pipeline integriert, um eine frühzeitige Fehlererkennung zu ermöglichen.

Die Erweiterungen verfolgen das Ziel, die Benutzerverwaltung zu automatisieren und gleichzeitig die Qualitätssicherung durch systematische Tests zu erhöhen. Dabei wird die bestehende Architektur bewusst nur minimal angepasst und auf Wiederverwendbarkeit sowie Konfigurierbarkeit geachtet.

11.2 Komponentendesign: Erweiterung von updateUser

Die überarbeitete Version der updateUser-Funktion wurde im Rahmen der Design-Phase neu konzipiert und in einem BPMN-ähnlichen Ablaufdiagramm visualisiert (siehe Abbildung 11.1). Im Vergleich zum bisherigen Zustand wurde der Ablauf erweitert und strukturiert automatisiert. Neu wird der Benutzer aus allen bestehenden Gruppen entfernt, um eine fehlerfreie Neuzuordnung zu ermöglichen.

Die zentrale Neuerung besteht in der rollenbasierten Gruppenzuweisung: Anhand der im Payload enthaltenen igRoles und eines in der Konfiguration definierten Mappings wird der Benutzer automatisch den korrekten Gruppen (z.B. BROKERPORTAL, VERTRAGSSERVICE) hinzugefügt. Abschliessend wird die IDP-Nummer im LDAP aktualisiert. Fehlerhafte oder unvollständige Anfragen führen in einen klar definierten Fehlerpfad mit entsprechender Fehlermeldung.

Die neue Struktur ist robuster, besser wartbar und reduziert manuelle Eingriffe erheblich.

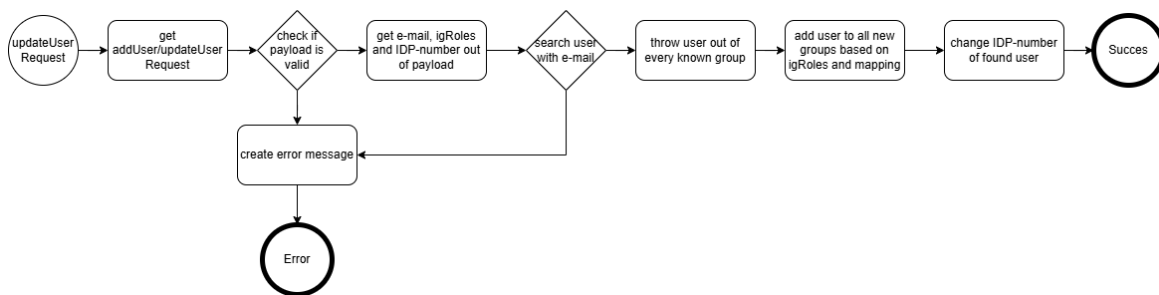


Abbildung 11.1: BPMN-Diagramm der updateUser-Funktion (Quelle: Eigene Darstellung)

11.2.1 Konfigurationsdesign: YAML-Mapping

Um eine zentrale und flexible Steuerung der Gruppenzuweisung im LDAP-Verzeichnis zu ermöglichen, wird ein rollenbasiertes Mapping über die Konfigurationsdatei `application.yml` realisiert. Diese Konfiguration erlaubt es, bestimmte Rollenpräfixe, wie sie im SOAP-Payload unter `igRoles` übermittelt werden, vordefinierten Gruppen im LDAP-Verzeichnis zuzuordnen.

Die Gruppen befinden sich unter dem in der Konfiguration definierten `groupSearchBase ou=groups,dc=paxportal,dc=ch`. Für jede Rollenfamilie (z. B. `EL_` für Einzelleben oder `KL_` für Kollektivleben) ist eine Liste von Gruppen hinterlegt, in die Benutzer automatisch aufgenommen werden, sofern die entsprechende Rolle im Payload enthalten ist.

Die Abkürzungen EL und KL orientieren sich an der fachlichen Struktur der Pax:

- **EL** steht für *Einzelleben* und entspricht der *Privaten Vorsorge (PV)*.
- **KL** steht für *Kollektivleben* und entspricht der *Beruflichen Vorsorge (BV)*.

Die farbliche Markierung der Gruppen in Abbildung 11.2 veranschaulicht die Zuordnung:

- Gelb markierte Gruppen gehören zur Kategorie **EL**.

- Rot markierte Gruppen gehören zur Kategorie **KL**.

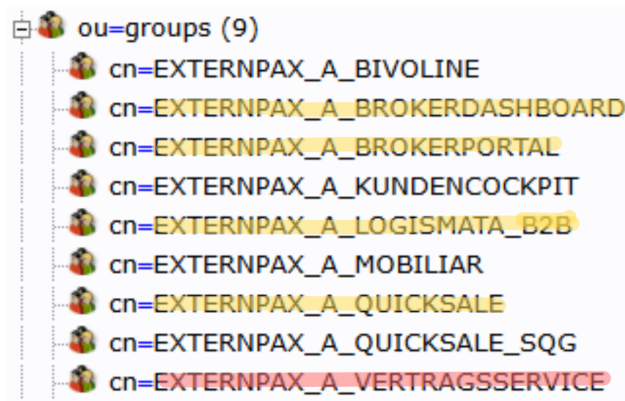


Abbildung 11.2: LDAP-Gruppen mit farblicher Zuordnung zu EL (gelb) und KL (rot)

Nachfolgend ein exemplarischer Auszug aus der Konfiguration:

Listing 11.1: Auszug aus der YAML-Konfiguration für rollenbasiertes Mapping

```
ecohub:
  ldap:
    groupSearchBase: ou=groups,dc=paxportal,dc=ch
    roleMappings:
      EL_:
        - cn=EXTERNPAX_A_BROKERPORTAL,ou=groups
        - cn=EXTERNPAX_A_QUICKSALE,ou=groups
        - cn=EXTERNPAX_A_LOGISMATA_B2B,ou=groups
        - cn=EXTERNPAX_A_BROKERDASHBOARD,ou=groups
      KL_:
        - cn=EXTERNPAX_A_VERTRAGSSERVICE,ou=groups
```

Alle weiteren Gruppen, die in der Darstellung ersichtlich sind, aber nicht in der YAML-Konfiguration aufgeführt werden, stehen aktuell in keinem direkten Zusammenhang mit den für dieses Projekt relevanten Applikationen und wurden daher im Rahmen des Designs nicht berücksichtigt. Das Mapping kann jedoch jederzeit erweitert werden, sollte sich dies in Zukunft ändern.

11.3 Testdesign und API-Abdeckung

Zur Sicherstellung der Funktionalität und zur frühzeitigen Fehlererkennung werden für alle vier vorhandenen SOAP-Endpunkte systematische API-Tests eingeführt. Die Tests werden mit dem Framework **Playwright** entwickelt, welches durch seine Flexibilität sowohl für Web- als auch für API-Tests geeignet ist.

Zielsetzung

- Automatisierte Tests für: addUser, updateUser, deleteUser, getUserStatus
- Prüfung auf korrekte Verarbeitung valider Payloads
- Erkennung von Fehlerfällen bei ungültigen oder unvollständigen Eingaben
- Integration in die bestehende GitHub-Actions-Pipeline

11.4 Testfälle

Die Tests werden im gleichen Repository wie die Anwendung selbst gehalten. Es wird ein separater Ordner für die Tests erstellt. In der CI/CD-Pipeline wird ein dedizierter Job integriert, der die Tests bei jedem Build ausführt. So können fehlerhafte Implementierungen frühzeitig erkannt und rückgemeldet werden.

11.4.1 Vollständige XML-Payloads zu addUser, updateUser

Die Testfälle zeigen nur die wichtigsten Daten der Anfragen. Folgend daher die einmal die vollständigen XML-Payloads für die zwei Endpunkte.

Listing 11.2: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>-999</idpUserId>
      <orgId>1</orgId>
      <igRoles>EL_OFFERT</igRoles>
      <orgRegNo>10</orgRegNo>
      <primaryOU>
        <id>1</id>
        <regNo>10</regNo>
        <type>test</type>
        <name>test</name>
      </primaryOU>
      <employed>true</employed>
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei@pax.ch</userEmail>
      <userOfficePhone>123</userOfficePhone>
      <userMobilePhone>123</userMobilePhone>
      <userBirthday>1970-01-01T00:00:00Z</userBirthday>
      <userSex>M</userSex>
      <userLanguage>DE</userLanguage>
      <validFrom>2024-01-01T00:00:00Z</validFrom>
      <validTo>2024-12-31T00:00:00Z</validTo>
      <userType>PERSONAL</userType>
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<userCategory>BROKER</userCategory>
<creationDate>2024-01-01T00:00:00Z</creationDate>
<creationUser>admin</creationUser>
<modificationDate>2024-01-01T00:00:00Z</modificationDate>
<modificationUser>admin</modificationUser>
<requestId>1</requestId>
<requestTime>2024-01-01T00:00:10Z</requestTime>
  </user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Nun die Testfälle zu den einzelnen Endpunkten:

11.4.2 Endpoint: addUser

Test-ID: 1

Beschreibung:

addUser Nutzer erhält erfolgreich eine IDP-Nummer.

Input:

Listing 11.3: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>100</idpUserId>
      ...
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei@pax.ch</userEmail>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.4: addUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonResponseType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonResponseType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



Test-ID: 2

Beschreibung:

addUser Nutzer hat bereits eine IDP-Nummer.

Input:

Listing 11.5: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>101</idpUserId>
      ...
      <givenName>Max</givenName>
      <surname>Mustermann</surname>
      <userEmail>max.mustermann@pax.ch</userEmail>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.6: addUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_EXISTS</returnCode>
      <errorMessage>User already exists with idpUserId=101</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 3

Beschreibung:

addUser Payload unvollständig.

Input:

Listing 11.7: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>103</idpUserId>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<givenName>Marco</givenName>

<userEmail>marco.frei@pax.ch</userEmail>
...
</user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.8: addUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Payload validation failed for idpUserId=111</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 4

Beschreibung:

addUser Email-Adresse kann nicht gefunden werden.

Input:

Listing 11.9: addUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:addUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>104</idpUserId>
      ...
      <givenName>Marco</givenName>
      <surname>Frei</surname>
      <userEmail>marco.frei1@bug.ch</userEmail>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.10: addUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <returnCode>ERROR</returnCode>
  <errorMessage>Processing add user failed for idpUserId=104</errorMessage>
</ns3:commonReturnType>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.3 Endpoint: updateUser

Test-ID: 5

Beschreibung:

updateUser Nutzer erhält erfolgreich eine neue IDP-Nummer.

Input:

Listing 11.11: updateUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>200</idpUserId>
      ...
      <givenName>Heidi</givenName>
      <surname>Meier</surname>
      <userEmail>heidi.meier@pax.ch</userEmail>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.12: updateUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 6

Beschreibung:

updateUser Gesuchter Nutzer nicht gefunden.

Input:

Listing 11.13: updateUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>201</idpUserId>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
...
    <givenName>Heidi</givenName>
    <surname>Meier</surname>
    <userEmail>heidi.meier1@bug.ch</userEmail>
    ...
  </user:addUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.14: updateUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User cannot update for idpUserId=201</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 7

Beschreibung:

updateUser Payload unvollständig.

Input:

Listing 11.15: updateUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/
  userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:updateUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>111</idpUserId>
      ...
      <givenName>Marco</givenName>

      <userEmail>marco.frei@pax.ch</userEmail>
      ...
    </user:addUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.16: updateUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<SOAP-ENV:Body>
  <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
    <returnCode>ERROR</returnCode>
    <errorMessage>Payload validation failed for idpUserId=111</errorMessage>
  </ns3:commonReturnType>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.4 Endpoint: deleteUser

Test-ID: 8

Beschreibung:

deleteUser IDP-Nummer wird erfolgreich gelöscht.

Input:

Listing 11.17: deleteUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>005</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
      <deletionUser>admin</deletionUser>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:10Z</requestTime>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.18: deleteUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 9

Beschreibung:

deleteUser Payload unvollständig.

Input:

Listing 11.19: deleteUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>11111111111111111111111111111111</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<deletionUser>admin</deletionUser>
<requestId>1</requestId>
<requestTime>2024-12-01T00:00:10Z</requestTime>
</user:deleteUserType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.20: deleteUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>ERROR</returnCode>
      <errorMessage>Payload validation for delete user failed for idpUserId=11111111111111111111</errorMessage>
    </ns3:commonReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 10

Beschreibung:

deleteUser Nutzer nicht gefunden.

Input:

Listing 11.21: deleteUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:deleteUserType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>999</idpUserId>
      <deletionDate>2024-12-01T00:00:00Z</deletionDate>
      <deletionUser>admin</deletionUser>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:10Z</requestTime>
    </user:deleteUserType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.22: deleteUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:commonReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
<returnCode>USER_NOT_FOUND</returnCode>
  <errorMessage>User to be deleted not found idpUserId=999</errorMessage>
</ns3:commonResponseType>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.5 Endpoint: getUser

Test-ID: 11**Beschreibung:**

getUser Nutzer wird erfolgreich gefunden und IDP-Nummer existiert.

Input:

Listing 11.23: getUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:getUserStatusType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>004</idpUserId>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:00Z</requestTime>
    </user:getUserStatusType>
  </soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.24: getUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:userStatusReturnTypes xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>OK</returnCode>
      <userStatus>OK</userStatus>
    </ns3:userStatusReturnTypes>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Test-ID: 12**Beschreibung:**

getUser Nutzer nicht gefunden.

Input:

Listing 11.25: getUser-Anfrage mit fehlerhaften Daten Input

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
  <soapenv:Header/>
  <soapenv:Body>
    <user:getUserStatusType xmlns:user="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <idpUserId>999</idpUserId>
      <requestId>1</requestId>
      <requestTime>2024-12-01T00:00:00Z</requestTime>
    </user:getUserStatusType>
  </soapenv:Body>
</soapenv:Envelope>
```


Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



```
</user:getUserStatusType>
</soapenv:Body>
</soapenv:Envelope>
```

Erwartet:

Listing 11.26: getUser-Anfrage mit fehlerhaften Daten Output

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <SOAP-ENV:Body>
    <ns3:userStatusReturnType xmlns:ns3="http://www.igb2b.ch/userProvisioningSchemaTypes-v3.0/">
      <returnCode>USER_NOT_FOUND</returnCode>
      <errorMessage>User status not found for idpUserId=999</errorMessage>
    </ns3:userStatusReturnType>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

11.4.6 Endpoint: addUser, updateUser, deleteUser, getUser

Sollte der Service nicht ansprechbar sein, wird jegliche Anfrage mit einem Fehler beantwortet.

Test-ID: 14

Beschreibung:

addUser, updateUser, deleteUser, getUser Service ist nicht erreichbar.

Input:

Listing 11.27: getUser-Anfrage mit fehlerhaften Daten Input

EGAL

Erwartet:

Listing 11.28: getUser-Anfrage mit fehlerhaften Daten Output

null/empty

11.4.7 Erweiterung der CI/CD-Pipeline mit Docker und Playwright

Zur Sicherstellung der korrekten Zusammenarbeit zwischen der Applikation und dem LDAP-System ist geplant, einen zusätzlichen CI/CD-Job für automatisierte End-to-End-Tests (E2E) zu implementieren. Dieser basiert auf dem bestehenden `docker-compose.yml` und nutzt Playwright zur Ausführung der Tests.

Hinweis: Es handelt sich hierbei um eine konzeptionelle Designüberlegung. Die finale technische Umsetzung kann im Verlauf der Implementierungsphase angepasst oder erweitert werden.

Geplante Testumgebung

Mittels Docker Compose soll eine isolierte Testumgebung aufgesetzt werden, die folgende Komponenten umfasst:

- Die Spring-Boot-Applikation (`pax-ecohub-provisioning`)
- Einen OpenLDAP-Container mit vordefiniertem Schema und Testdaten

Ablauf des neuen Jobs `api-tests`

1. Start der Testumgebung mit `docker-compose up -d`
2. Warten auf Initialisierung der Dienste
3. Ausführen der Playwright-Tests gegen die laufende Umgebung
4. Aufräumen der Umgebung mittels `docker-compose down`

Geplanter CI/CD-Ausschnitt

Listing 11.29: Entwurf: CI/CD-Job für E2E-Tests

```
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Docker Compose environment
        run: docker-compose up -d

      - name: Wait for services to initialize
        run: sleep 20

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies and run Playwright tests
        run: |
          npm ci
          npx playwright test

      - name: Tear down environment
        if: always()
        run: docker-compose down
```

Ziele dieser Erweiterung

- Vollständige Integrationstests der Anwendung in Kombination mit LDAP
- Sicherstellung der API-Funktionalität unter realen Bedingungen
- Wiederverwendung der vorhandenen Container-Infrastruktur aus der Entwicklung

11.5 Technische Arbeitspakete

In folgendem Abschnitt werden die technischen Arbeitspakete aufgelistet, die im Rahmen dieses Projekts umgesetzt werden. Es handelt sich hierbei um veränderungen an dem bestehenden Code, keine dokumentationen oder organisationellen Änderungen.

Da die Mapping-Funktion Änderungen an der bestehenden Struktur erfordert, wird diese zuerst umgesetzt.

Tabelle 11.1: Technische Arbeitspakete

ID	Titel	Beschreibung	Dauer
1	Konfigurationsfile erstellen	Es wird ein Konfigurationsfile erstellt, welches die Mapping-Funktion definiert.	1h
2	Entferne Nutzer aus bekannten Gruppen	Es wird eine Funktion implementiert, welche die Nutzer aus den bekannten Gruppen entfernt.	1h
3	Nutzer den neuen Gruppen zuordnen	Es wird eine Funktion implementiert, welche die Nutzer den neuen Gruppen zuordnet.	2h
4	Testen der updateUser-Funktion	Die Funktion wird getestet, ob sie die Mapping-Funktion korrekt umsetzt.	2h
5	API-Tests für Endpunkt addUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes addUser testet.	1h
6	API-Tests für Endpunkt updateUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes updateUser testet.	1h
7	API-Tests für Endpunkt deleteUser erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes deleteUser testet.	1h
8	API-Tests für Endpunkt getUserStatus erstellen	Es wird eine Playwright-Testsuite erstellt, welche die API-Funktionalität des Endpunktes getUserStatus testet.	1h
9	Integration der Tests in die CI/CD-Pipeline	Es wird die Testsuite in die CI/CD-Pipeline integriert und die Tests werden ausgeführt.	2h

11.6 Tools und Technologien

In diesem Abschnitt werden die im Projekt eingesetzten Tools und Technologien beschrieben. Sie dienen der Unterstützung bei Entwicklung, Test und Qualitätssicherung der erweiterten SOAP-Schnittstellen sowie der LDAP-Anbindung.

11.6.1 Testen der Mapping-Funktion mit SoapUI und OpenLDAP

Die im Rahmen der updateUser-Funktion implementierte rollenbasierte Mapping-Funktion kann nicht vollständig durch automatisierte API-Tests abgedeckt werden. Um diese dennoch zuverlässig zu testen, wird ein manueller Testansatz verfolgt.

Hierzu wird die Anwendung lokal gestartet und mit Hilfe von **SoapUI** gezielt über den updateUser-SOAP-Endpunkt angesprochen. Gleichzeitig wird ein **OpenLDAP**-Container lokal ausgeführt, welcher als Testver-

zeichnisdienst dient. So kann überprüft werden, ob Benutzer – basierend auf übermittelten Rollen – korrekt den entsprechenden Gruppen im LDAP hinzugefügt werden.

SoapUI ermöglicht das flexible Erstellen und Anpassen von SOAP-Anfragen, wodurch verschiedene Rollenkonstellationen getestet werden können. Durch anschließende Kontrolle der Einträge im LDAP (z. B. mit `ldapsearch`) lässt sich die Funktionsweise der Mapping-Logik gezielt überprüfen.

11.6.2 Automatisierte API-Tests mit Playwright

Zur Sicherstellung der allgemeinen Funktionsfähigkeit der implementierten SOAP-Endpunkte wird das Framework **Playwright** eingesetzt. Obwohl Playwright ursprünglich für browserbasierte End-to-End-Tests konzipiert wurde, lässt es sich auch hervorragend für HTTP-basierte API-Tests verwenden.

In diesem Projekt wird Playwright verwendet, um die vier zentralen Funktionen der SOAP-Schnittstelle automatisiert zu testen: `addUser`, `updateUser`, `deleteUser` und `getUserStatus`. Ziel ist es, die korrekte Verarbeitung valider Anfragen zu prüfen sowie typische Fehlerfälle (z. B. fehlende Felder oder ungültige Inhalte) automatisiert zu erkennen. Die Tests werden in TypeScript implementiert und als Teil der CI/CD-Pipeline über GitHub Actions ausgeführt.

Erfahrung im Unternehmen: Bei der **PAX** wird Playwright bereits erfolgreich in anderen Entwicklungsprojekten eingesetzt, insbesondere zur automatisierten Prüfung von Webanwendungen. Die vorhandene Erfahrung im Team sowie bestehende Best Practices sprachen für den Einsatz dieses Tools auch im Rahmen dieses Projekts.

Vergleich mit Alternativen: Vor der Entscheidung für Playwright wurden auch andere Tools in Betracht gezogen, wie etwa:

- **Postman/Newman:** Gute Unterstützung für einfache REST-Tests, aber eingeschränkte SOAP-Funktionalität und weniger flexible Automatisierung.
- **REST Assured (Java):** Sehr leistungsfähig für HTTP-Tests, allerdings eng an die Java-Welt gebunden und weniger etabliert im Team.
- **SoapUI:** Gut geeignet für manuelle Tests, aber weniger flexibel für API-Testautomatisierung in CI/CD.

Die Entscheidung fiel letztlich zugunsten von Playwright, da es bereits im Unternehmen etabliert ist, moderne Entwicklungs-Workflows unterstützt und sich gut in bestehende Pipelines integrieren lässt.

11.6.3 Versionierung

Eine sichere Versionsverwaltung ist besonders wichtig, sollten Daten verloren gehen, kann diese Sicherung die Daten wiederherstellen.

In dieser Arbeit wird für die Versionierung Git mit der Online-Plattform Github verwendet, da die Pax bereits Github als VCS verwendet.

Git ist ein verteiltes Versionskontrollsystem, mit dem Entwickler den Verlauf von Änderungen an Dateien insbesondere Quellcode nachverfolgen können. Es ermöglicht mehrere parallele Entwicklungszweige, erleichtert das Zusammenführen von Änderungen und unterstützt das Arbeiten im Team ohne Datenverlust.

GitHub ist die dazu passende Online-Plattform, die Git-Repositories hostet. Sie bietet zusätzliche Funktionen wie Benutzeroberfläche, Pull Requests, Issue-Tracking und CI/CD-Integration. GitHub erleichtert die Zusammenarbeit an Softwareprojekten und macht den Code öffentlich oder privat zugänglich.

11.7 Clean Code Prinzipien und Namenskonzept

Im Rahmen der Implementierung wurde besonderer Wert auf die Einhaltung von **Clean Code Prinzipien** gelegt. Ziel war es, die Lesbarkeit, Verständlichkeit und Wartbarkeit des Codes sicherzustellen. Die Umsetzung orientierte sich an bewährten Best Practices nach Robert C. Martin („Uncle Bob“) sowie an internen Entwicklungsrichtlinien der PAX.

Die wichtigsten Massnahmen zur Verbesserung der Codequalität umfassten:

- **Sprechende Bezeichner:** Methoden, Variablen und Klassen wurden mit aussagekräftigen Namen versehen, um deren Funktion klar zu kommunizieren (z. B. `mapRolesToGroups()` statt `handle()`).
- **Namenskonzept und Konventionen:** Es wurde ein einheitliches Namenskonzept verwendet:
 - camelCase für Variablen- und Methodennamen (z. B. `userId`, `updateUserEntry()`),
 - PascalCase für Klassennamen (z. B. `PersonRepositoryImpl`),
 - UPPER_SNAKE_CASE für Konstanten (z. B. `LDAP_BASE_DN`).

Diese Konventionen fördern die visuelle Trennung und helfen dabei, unterschiedliche Code-Elemente auf einen Blick zu erkennen.

- **Single Responsibility Principle:** Methoden und Klassen wurden so gestaltet, dass sie jeweils nur eine klar definierte Aufgabe übernehmen. Komplexe Logik wurde in kleinere, gut testbare Einheiten aufgeteilt.
- **Vermeidung von magischen Werten:** Feste Werte wie Gruppen-DNs oder Rollenpräfixe wurden zentral in der `application.yml` oder in Konstanten abgelegt.
- **Konsistente Formatierung:** Der Quellcode wurde gemäss dem PAX-internen Formatierungsstil (basierend auf dem Google Java Style Guide) strukturiert. Dies wurde durch entsprechende IDE-Tools automatisiert unterstützt.
- **Zielgerichtete Fehlerbehandlung:** Fehlerfälle wurden gezielt behandelt und mit verständlichen Logmeldungen versehen, um eine einfache Diagnose und Debugging zu ermöglichen.

Die konsequente Anwendung dieser Prinzipien verbessert nicht nur die Wartbarkeit, sondern erleichtert auch den Wissenstransfer innerhalb des Teams. Neue Entwicklerinnen und Entwickler können sich so schneller im Code zurechtfinden.

KAPITEL 12

IMPLEMENTIERUNG

Das folgende Kapitel wurde in zwei grundlegende Abschnitte unterteilt.

Im ersten Abschnitt wird die Mapping-Funktion behandelt, es wird veranschaulicht, welche neuen Methoden hinzugefügt wurden und wie die bisherigen Methoden verändert wurden. Alle neuen Dateien werden erklärt und mit einem kurzen Code-Schnipsel veranschaulicht.

Im zweiten Abschnitt wird die Testautomation behandelt. Es wird erklärt, wie die einzelnen Endpunkte getestet werden, wie die Tests aufgebaut sind und wie diese automatisiert werden.

12.1 Mapping-Funktion

Die rollenbasierte Gruppenzuweisung ist ein zentraler Bestandteil der Benutzerverwaltung im Rahmen dieser Erweiterung. Sie sorgt dafür, dass Benutzer auf Basis ihrer übermittelten Rollen automatisch den richtigen Gruppen im LDAP-Verzeichnis zugewiesen werden. Die dafür notwendige Konfiguration sowie die Implementierung der entsprechenden Funktionen werden im Folgenden beschrieben.

12.1.1 Konfigurationsdatei

Die Konfigurationsdatei für die Mapping-Funktion befindet sich im Projekt unter dem Namen `application.yml`. In dieser Datei werden die Rollen Kürzel (z. B. `EL_`, `KL_`) den zugehörigen LDAP-Gruppen zugewiesen.

Listing 12.1: Auszug aus der YAML-Konfiguration für rollenbasiertes Mapping

```
ecohub:
  ldap:
    groupSearchBase: ou=groups,dc=paxportal,dc=ch
    roleMappings:
      EL_:
```

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



- cn=EXTERNPAX_A_BROKERPORTAL,ou=groups
- cn=EXTERNPAX_A_QUICKSALE,ou=groups
- cn=EXTERNPAX_A_LOGISMATA_B2B,ou=groups
- cn=EXTERNPAX_A_BROKERDASHBOARD,ou=groups

KL_:

- cn=EXTERNPAX_A_VERTRAGSSERVICE,ou=groups

Die Datei liegt im Ressourcenverzeichnis des Projekts, wie in Abbildung 12.1 dargestellt.

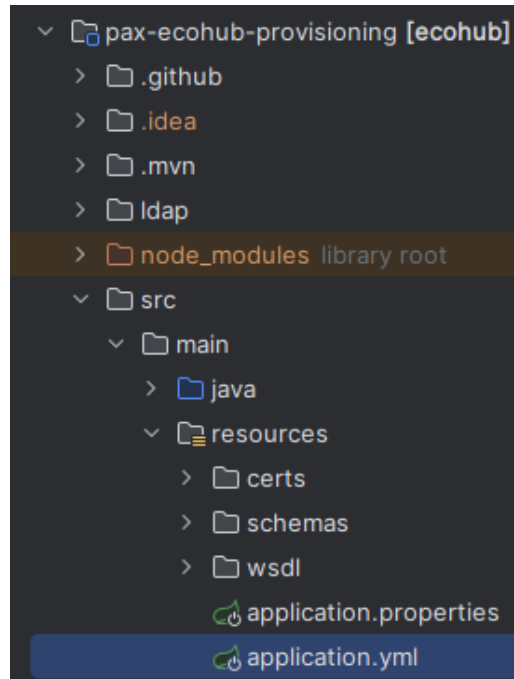


Abbildung 12.1: Pfad zur application.yml-Datei

12.1.2 Entfernen des Nutzers aus bestehenden Gruppen

Bevor ein Nutzer auf Basis seiner neuen Rollen den zugehörigen Gruppen zugeordnet werden kann, muss er aus allen Gruppen entfernt werden, in denen er möglicherweise noch eingetragen ist. Dies stellt sicher, dass keine veralteten oder mehrfachen Gruppenzugehörigkeiten bestehen.

Zweck der Methode

Die Methode `removeUserFromAllGroup` entfernt einen Nutzer anhand seines sogenannten *Distinguished Name (DN)* aus allen Gruppen, die über das Rollenkonfigurations-Mapping bekannt sind. Die Methode befindet sich in der Datei `PersonRepositoryImpl.java`.

Aufbau der Methode

Die Methode ist in mehrere sinnvolle Teilschritte gegliedert, die im Folgenden einzeln erklärt werden.

1. Zusammensetzen des vollständigen Benutzer-DN Damit der Nutzer in den LDAP-Gruppen eindeutig identifiziert werden kann, muss sein DN vollständig angegeben werden. Dazu wird an den mitgegebenen DN-Teil der Domänenteil `dc=paxportal,dc=person` angehängt:

Listing 12.2: Konstruktion des vollständigen Benutzer-DN

```
userDn = userDn + "dc=paxportal,dc=person";
```

2. Ermittlung aller bekannten Gruppen Statt alle Gruppen manuell aufzulisten, wird hier dynamisch aus der bestehenden Rollenkonfiguration ('roleMappings') ermittelt, welche Gruppen existieren. So ist die Methode flexibel erweiterbar, ohne Änderungen im Code:

Listing 12.3: Dynamisches Sammeln aller Zielgruppen

```
Set<String> allGroups = new HashSet<>();  
for (List<String> groups : getRoleMappings().values()) {  
    allGroups.addAll(groups);  
}
```

3. Entfernen des Nutzers aus jeder Gruppe Für jede gefundene Gruppe wird versucht, den Nutzer aus dem Gruppenobjekt zu entfernen. Dies geschieht über eine LDAP-Änderungsoperation, bei der der 'member'-Eintrag des Nutzers gelöscht wird. Falls der Nutzer nicht in der Gruppe enthalten ist oder ein Fehler auftritt, wird dies protokolliert, aber die Ausführung geht weiter:

Listing 12.4: LDAP-Entfernungsvorgang für Gruppenmitgliedschaft

```
for (String groupDn : allGroups) {  
    try {  
        ModificationItem[] mods = new ModificationItem[] {  
            new ModificationItem(DirContext.REMOVE_ATTRIBUTE, new BasicAttribute("member", userDn))  
        };  
  
        ldapTemplate.modifyAttributes(groupDn, mods);  
        log.info("Remove user from group [userDn={}, groupDn={}]", userDn, groupDn);  
    } catch (Exception e) {  
        log.debug("User not in group or other error. [userDn={}, groupDn={}, exception={}]",  
            userDn, groupDn, e.getMessage());  
    }  
}
```

4. Rückmeldung über Erfolg oder Fehler Zum Schluss gibt die Methode einen `boolean`-Wert zurück. Dieser signalisiert, ob der gesamte Entfernungsvorgang erfolgreich war oder ob ein Fehler auftrat:

Listing 12.5: Fehlerbehandlung und Rückgabewert

```
log.info("Remove user from group [userDn={}]", userDn);
return true;
} catch (Exception e) {
    log.error("Failed to remove user from groups. [userDn={}, exception={}]", userDn, e.getMessage());
    return false;
}
```

Fazit

Die Methode `removeUserFromAllGroup` sorgt für einen sauberen Ausgangszustand, indem sie alle bestehenden Gruppenzugehörigkeiten eines Nutzers entfernt. Damit wird sichergestellt, dass nachfolgende Zuweisungen (siehe nächste Subsection) konsistent und fehlerfrei erfolgen können. Durch die dynamische Ermittlung der Gruppen ist die Methode flexibel und wartungsarm.

Die komplette removeUserFromAllGroup Methode

Hier die komplette removeUserFromAllGroup Methode:

Listing 12.6: Die komplette removeUserFromAllGroup Methode

```
private boolean removeUserFromAllGroup(String userDn) {
    userDn = userDn + "dc=paxportal,dc=person";
    try {
        Set<String> allGroups = new HashSet<>();
        for (List<String> groups : getRoleMappings().values()) {
            allGroups.addAll(groups);
        }

        for (String groupDn : allGroups) {
            try {
                ModificationItem[] mods = new ModificationItem[] {
                    new ModificationItem(DirContext.REMOVE_ATTRIBUTE, new BasicAttribute("member", userDn))
                };

                ldapTemplate.modifyAttributes(groupDn, mods);
                log.info("Remove user from group [userDn={}, groupDn={}]", userDn, groupDn);
            } catch (Exception e) {
                log.debug("User not in group or other error. [userDn={}, groupDn={}, exception={}]",
                    userDn, groupDn, e.getMessage());
            }
        }

        log.info("Remove user from group [userDn={}]", userDn);
        return true;
    } catch (Exception e) {
        log.error("Failed to remove user from groups. [userDn={}, exception={}]", userDn, e.getMessage());
        return false;
    }
}
```

12.1.3 Hinzufügen des Nutzers zu den neuen Gruppen

Nachdem der Nutzer aus allen bestehenden LDAP-Gruppen entfernt wurde (siehe Abschnitt ??), wird er in einem zweiten Schritt wieder den richtigen Gruppen zugewiesen. Dies geschieht auf Basis der Rollen, die im sogenannten CompleteUserType-Objekt übermittelt werden. Die zentrale Methode für diese Aufgabe heisst updateUserGroup und befindet sich in der Datei PersonRepositoryImpl.java.

Zweck der Methode

Die Methode analysiert die Rollen des Nutzers (igRoles), vergleicht sie mit der Konfiguration aus der application.yml, und fügt den Nutzer dann den passenden Gruppen im LDAP hinzu.

Aufbau der Methode

Im Folgenden werden die wichtigsten Bestandteile der Methode schrittweise erklärt und jeweils mit einem Codeausschnitt ergänzt.

1. Vorbereitung und Abbruch bei fehlenden Rollen Zu Beginn wird überprüft, ob der Nutzer überhaupt Rollen besitzt. Falls keine vorhanden sind, wird der Vorgang übersprungen:

Listing 12.7: Abbruch bei fehlenden Rolleninformationen

```
if (payload == null || payload.getIgRoles() == null || payload.getIgRoles().isEmpty()) {  
    log.info("No igRoles provided, skipping group updates");  
    return;  
}
```

2. Ermittlung der Zielgruppen basierend auf Rollenpräfixen Es wird eine Liste von Zielgruppen aufgebaut. Dafür werden alle Rollen mit den definierten Präfixen (z. B. EL_, KL_) verglichen. Jede passende Rolle führt zur Zuordnung der zugehörigen Gruppen:

Listing 12.8: Zuordnung der Rollen zu Gruppen

```
Map<String, Boolean> targetGroups = new HashMap<>();  
List<String> igRoles = payload.getIgRoles();  
  
for (String igRole : igRoles) {  
    for (Map.Entry<String, List<String>> entry : getRoleMappings().entrySet()) {  
        String prefix = entry.getKey();  
        List<String> groups = entry.getValue();  
  
        if (igRole.startsWith(prefix)) {  
            for (String groupDn : groups) {  
                targetGroups.put(groupDn, true);  
            }  
        }  
    }  
}
```

3. Nutzer wird zu jeder relevanten Gruppe hinzugefügt Für jede ermittelte Gruppe wird eine LDAP-Operation ausgeführt, die den Nutzer der Gruppe hinzufügt. Diese Operation wird mit einer sogenannten ModificationItem-Anweisung durchgeführt:

Listing 12.9: Hinzufügen des Nutzers zu den LDAP-Gruppen

```
for (String groupDn : targetGroups.keySet()) {  
    try {  
        ModificationItem[] mods = new ModificationItem[] {  
            new ModificationItem(DirContext.ADD_ATTRIBUTE, new BasicAttribute("member", userDn))  
        };  
    }
```

```
        ldapTemplate.modifyAttributes(groupDn, mods);
        log.info("Added user to group. [userDn={}, groupDn={}]", userDn, groupDn);
    } catch (Exception e) {
        log.error("Failed to add user to group. [userDn={}, groupDn={}, exception={}, cause={}]",
            userDn, groupDn, e.getMessage(), e.getCause() != null ? e.getCause().getMessage() : "none");
    }
}
```

4. Fehlerbehandlung Die gesamte Methode ist zusätzlich in einen try-catch-Block gehüllt, um bei allgemeinen Fehlern während der Gruppenverarbeitung ein Logging vorzunehmen:

Listing 12.10: Allgemeine Fehlerbehandlung am Ende der Methode

```
} catch (Exception e) {
    log.error("Failed to update user groups. [userDn={}, exception={}]", userDn, e.getMessage());
}
```

Fazit

Die Methode `updateUserGroup` ist zentral für die automatische rollenbasierte Gruppensteuerung im LDAP. Sie stellt sicher, dass ein Nutzer nach dem Update nur noch in den Gruppen enthalten ist, die seiner aktuellen Rollenstruktur entsprechen. Dabei wird auf Konfigurierbarkeit über die YAML-Datei gesetzt, wodurch die Lösung flexibel erweitert werden kann.

Die komplette updateUserGroup Methode

Hier die komplette updateUserGroup Methode:

Listing 12.11: Die komplette updateUserGroup Methode

```
private void updateUserGroup(String userDn, final CompleteUserType payload) {
    userDn = userDn + ",dc=paxportal,dc=ch";
    if (payload == null || payload.getIgRoles() == null || payload.getIgRoles().isEmpty()) {
        log.info("No igRoles provided, skipping group updates");
        return;
    }

    try {
        Map<String, Boolean> targetGroups = new HashMap<>();
        List<String> igRoles = payload.getIgRoles();

        for (String igRole : igRoles) {
            for (Map.Entry<String, List<String>> entry : getRoleMappings().entrySet()) {
                String prefix = entry.getKey();
                List<String> groups = entry.getValue();

                if (igRole.startsWith(prefix)) {
                    for (String groupDn : groups) {
                        targetGroups.put(groupDn, true);
                    }
                }
            }
        }

        for (String groupDn : targetGroups.keySet()) {
            try {
                ModificationItem[] mods = new ModificationItem[] {
                    new ModificationItem(DirContext.ADD_ATTRIBUTE, new BasicAttribute("member", userDn))
                };

                log.debug("Attempting to add user to group. [userDn={}, groupDn={}]", userDn, groupDn);
                ldapTemplate.modifyAttributes(groupDn, mods);
                log.info("Added user to group. [userDn={}, groupDn={}]", userDn, groupDn);
            } catch (Exception e) {
                log.error("Failed to add user to group. [userDn={}, groupDn={}, exception={}, cause={}]",
                    userDn, groupDn, e.getMessage(), e.getCause() != null ? e.getCause().getMessage() : "none");
            }
        }

        log.info("Remove user from group [userDn={}]", userDn);
    } catch (Exception e) {
        log.error("Failed to update user groups. [userDn={}, exception={}]", userDn, e.getMessage());
    }
}
```

12.1.4 Einbindung der Mapping-Funktionen in updateUser

Die zuvor beschriebenen Methoden `removeUserFromAllGroup` und `updateUserGroup` werden innerhalb der zentralen `updateUser`-Funktion aufgerufen. Diese Funktion wird verwendet, wenn bestehende Benutzer im LDAP-Verzeichnis aktualisiert werden sollen.

Dabei erfolgt die Gruppenverwaltung in drei aufeinanderfolgenden Schritten:

1. Änderung der Basisattribute des Benutzers (z. B. IDP-Nummer)
2. Entfernen des Benutzers aus allen Gruppen
3. Neue Gruppenzuweisung basierend auf den aktuellen Rollen

Die folgende Methode zeigt die vollständige Integration:

Listing 12.12: updateUser-Methode mit eingebauten Mapping-Funktionen

```
@Override
public boolean updatePerson(final Person person, final CompleteUserType payload) {
    final String idpUserId = payload.getIdpUserId();
    boolean updated = false;
    log.info("Update person in LDAP started. [idpUserId={}]", idpUserId);

    try {
        ldapTemplate.modifyAttributes(person.getDn(), buildModification(idpUserId));

        boolean removalSuccess = removeUserFromAllGroup(person.getDn().toString());
        if (!removalSuccess) {
            log.error("Failed to remove user from groups. [idpUserId={}]", idpUserId);
            return false;
        }

        updateUserGroup(person.getDn().toString(), payload);

        log.info("Update person in LDAP succeeded. [idpUserId={}]", idpUserId);
        updated = true;

    } catch (final Exception e) {
        log.info("Update person in LDAP failed. [idpUserId={}, exception={}]", idpUserId, e.getMessage());
    }

    log.info("Update person in LDAP finished. [idpUserId={}]", idpUserId);
    return updated;
}
```

Erläuterung der Abläufe

- `modifyAttributes(...)`: Aktualisiert z. B. die IDP-Nummer des Nutzers im LDAP.
- `removeUserFromAllGroup(...)`: Entfernt den Nutzer aus allen Gruppen, basierend auf dem aktuellen Rollenkontext.

- `updateUserGroup(...)`: Fügt den Nutzer anhand der Rollen (`igRoles`) den in der YAML-Konfiguration definierten Gruppen hinzu.

Diese Einbettung stellt sicher, dass jeder Benutzer nach einem Update nur genau in den Gruppen enthalten ist, die seiner aktuellen Rolle entsprechen.

12.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 13

TEST/QUALITÄTSSICHERUNG

Beschreibung der Test- und Qualitätssicherungsphase.

13.1 Mapping-Funktionen

Beschreibung der Mapping-Funktionen.

13.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 14

ERGEBNISSE

Beschreibung der erzielten Ergebnisse.

14.1 Mapping-Funktionen

Beschreibung der Mapping-Funktionen.

14.2 Testautomation

Beschreibung der Testautomation.

KAPITEL 15

DISKUSSION

Diskussion der Ergebnisse und Erkenntnisse.

15.1 Vergleiche

Beschreibung der Vergleiche.

Teil III

Anhang

HILFSMITTELVERZEICHNIS

- Hilfsmittel 1
- Hilfsmittel 2

ABKÜRZUNGSVERZEICHNIS

IPA Individuelle praktische Arbeit

GLOSSAR

- Begriff 1: Definition
- Begriff 2: Definition

ABBILDUNGSVERZEICHNIS

Abbildungsverzeichnis

2.1	Wasserfallmodell (Quelle: Eigenerarbeitung)	8
3.1	Projektaufbauorganisation (Quelle: Eigenerarbeitung)	11
6.1	Rollenmodell auf der EcoHub-Plattform (Quelle: Eigene Darstellung)	16
10.1	Rollenmodell auf der EcoHub-Plattform (Quelle: Miro Board)	22
10.2	LDAP-Verzeichnisstruktur im lokalen Setup (Quelle: Eigene Darstellung)	23
10.3	Business Process Model des IST-Zustands der Benutzerregistrierung (Quelle: Eigene Darstellung)	24
10.4	Systemkontext (C1) - IST-Zustand der Benutzerverwaltung (Quelle: Eigene Darstellung)	29
10.5	C2-Diagramm IST-Zustand - Containerübersicht (Quelle: Eigene Darstellung)	30
10.6	C3-Komponentendiagramm - EcoHub Provisioning Service (Quelle: Eigene Darstellung)	31
11.1	BPMN-Diagramm der updateUser-Funktion (Quelle: Eigene Darstellung)	37
11.2	LDAP-Gruppen mit farblicher Zuordnung zu EL (gelb) und KL (rot)	38
12.1	Pfad zur application.yml-Datei	58

TABELLENVERZEICHNIS

Tabellenverzeichnis

2.1 Vergleich zwischen Wasserfall und agilen Methoden 10

4.1 Aufgabenübersicht 12

10.1 Übersicht der zu testenden Endpunkte 24

11.1 Technische Arbeitspakete 54

Individuelle praktische Arbeit

Entwicklung einer Testautomation für die EcoHub-Plattform
und Implementierung von Mapping-Funktionen
Kandidat: Kento Puleo



QUELLENVERZEICHNIS